

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284427

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Thoppil; Anil Paul et al.

ZERO-COPY VOLUME MOVE WITHIN A DISTRIBUTED STORAGE SYSTEM

Abstract

Systems and methods for performing a zero-copy volume move between nodes of a distributed storage system are provided. In one example, an approach for performing a zero-copy volume move is proposed in which volume data may be maintained in place within a storage pod and need not be copied to move a given volume between the source node and the destination node. In one embodiment, file system metadata (e.g., a top-most physical volume block number (PVCN) of a node tree representing the volume at issue) of a write-anywhere file system is copied from the source node to the destination node. Since the storage pod is associated with a global PVCN space that is visible to all nodes of the distributed storage system, as a result of copying the top-mode PVCN of the volume to the destination node, anything below the top-most PVCN will automatically be visible to the destination node.

Inventors: Thoppil; Anil Paul (Pleasanton, CA), Subramanian; Ananthan (San Ramon, CA), Varghese; Kevin Daniel (Milpitas, CA), Choudhary; Garima (San Jose, CA), Zheng; Qinghua (San Jose, CA)

Applicant: NetApp, Inc. (San Jose, CA)

Family ID: 1000008393997

Assignee: NetApp, Inc. (San Jose, CA)

Appl. No.: 19/027436

Filed: January 17, 2025

Related U.S. Application Data

parent US continuation 18649906 20240429 parent-grant-document US 12204784 child US 19027436

parent US continuation-in-part 18595768 20240305 PENDING child US 18649906

Publication Classification

Int. Cl.: G06F3/06 (20060101); G06F16/13 (20190101)

U.S. Cl.:

CPC G06F3/0647 (20130101); G06F3/0608 (20130101); G06F3/0644 (20130101);
G06F3/0689 (20130101); G06F16/13 (20190101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 18/649,906, filed on Apr. 29, 2024, which is a continuation-in-part of U.S. patent application Ser. No. 18/595,768, filed on Mar. 5, 2024, both of which are hereby incorporated by reference in their entirety for all purposes.

BACKGROUND

Field

[0002] Various embodiments of the present disclosure generally relate to storage systems. In particular, some embodiments relate to the implementation and use of a storage pool, potentially having a disaggregated storage space, by a distributed storage system so as to facilitate, among other things, zero-copy volume movement from one node to another of the distributed storage system in which volume data is maintained in place within the storage pool and need not be copied to move a given volume between nodes of the distributed storage system, thereby allowing the transfer of the given volume to be performed as a constant-time operation independent of the size of the given volume.

Description of the Related Art

[0003] Distributed storage systems generally take the form of a cluster of storage controllers (or nodes in virtual or physical form). As a result of sub-optimal infrastructure architectures, prior scale-out storage solutions do not effectively utilize all three vectors of infrastructure (i.e., compute, network, and storage). For example, as shown in FIG. 5, each node of a distributed storage system may be associated with a dedicated pool of storage space (e.g., a node-level aggregate representing a file system that holds one or more volumes created over one or more RAID groups and which is only accessible from a single node at a time), thereby creating storage silos.

SUMMARY

[0004] Systems and methods are described for performing a zero-copy volume move between nodes of a distributed storage system. According to one embodiment, a storage pod is provided having a group of disks containing multiple Redundant Array of Independent Disks (RAID) groups in which an entirety of a global physical volume block number (PVCN) space associated with the storage pod is visible and accessible to all nodes of multiple nodes of a cluster representing a distributed storage system. A source volume is moved from a source node of the multiple nodes of the cluster to a destination node of the multiple nodes of the cluster without transferring underlying volume data of the source volume by copying file system metadata associated with a first container file representing the source volume to the destination node.

[0005] Other features of embodiments of the present disclosure will be apparent from accompanying drawings and detailed description that follows.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] In the Figures, similar components and/or features may have the same reference label. Further, various components of the same type may be distinguished by following the reference label with a second label that distinguishes among the similar components. If only the first reference label is used in the specification, the description is applicable to any one of the similar components having the same first reference label irrespective of the second reference label.

[0007] FIG. 1 is a block diagram illustrating a plurality of nodes interconnected as a cluster in accordance with an embodiment of the present disclosure.

[0008] FIG. 2 is a block diagram illustrating a node in accordance with an embodiment of the present disclosure.

[0009] FIG. 3 is a block diagram illustrating a storage operating system in accordance with an embodiment of the present disclosure.

[0010] FIG. 4 is a block diagram illustrating a tree of blocks representing of an example a file system layout in accordance with an embodiment of the present disclosure.

[0011] FIG. 5 is a block diagram illustrating a distributed storage system architecture in which the entirety of a given disk and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space.

[0012] FIG. 6A is a block diagram illustrating a distributed storage system architecture that provides disaggregated storage in accordance with an embodiment of the present disclosure.

[0013] FIG. 6B is a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod in accordance with an embodiment of the present disclosure.

[0014] FIG. 7A is a block diagram illustrating two dynamically extensible file systems (DEFSs) prior to performance of automatic space balancing in accordance with an embodiment of the present disclosure.

[0015] FIG. 7B is a block diagram illustrating the two dynamically extensible file systems (DEFSs) of FIG. 7A after completion of automatic space balancing in accordance with an embodiment of the present disclosure.

[0016] FIG. 8 is a flow diagram illustrating operations for performing space monitoring in accordance with an embodiment of the present disclosure.

[0017] FIG. 9 is a flow diagram illustrating operations for performing space balancing in accordance with an embodiment of the present disclosure.

[0018] FIG. 10 is a flow diagram illustrating operations for attempting to perform allocation area ownership change in support of space balancing in accordance with an embodiment of the present disclosure.

[0019] FIG. 11 is a flow diagram illustrating operations for performing space balancing responsive to removal of a node from a cluster in accordance with an embodiment of the present disclosure.

[0020] FIG. 12 is a flow diagram illustrating operations for performing space balancing responsive to addition of a new node to a cluster in accordance with an embodiment of the present disclosure.

[0021] FIG. 13A is a block diagram conceptually illustrating an initial state of a source node and a destination node prior to performance of a zero-copy volume move in accordance with an embodiment of the present disclosure.

[0022] FIG. 13B is a block diagram conceptually illustrating a state of the source node and the destination node of FIG. 13A after completion of the zero-copy volume move in accordance with an embodiment of the present disclosure.

[0023] FIG. 14 is a high-level flow diagram illustrating operations for performing zero-copy volume move in accordance with an embodiment of the present disclosure.

[0024] FIG. 15 is a more detailed flow diagram illustrating a distribution of operations between a

source node and a destination node for performing zero-copy volume move in accordance with an embodiment of the present disclosure.

DETAILED DESCRIPTION

[0025] Systems and methods are described for performing a zero-copy volume move between nodes of a distributed storage system. As compared to existing scale out storage solution architectures, various examples described herein facilitate various advantages, including, but not limited to, one or more of the following: [0026] Zero-copy volume move—in which volume data need not be copied to move a given volume between nodes of the distributed storage system, thereby facilitating the transfer of the given volume to be performed in constant time. [0027] Simplified management [0028] No silos of storage space [0029] Independent file system operation on each node of a cluster [0030] Independent scaling of performance and capacity (e.g., the ability to independently add nodes and/or disks on demand) [0031] Reduced internode (or “East-West”) communications/traffic [0032] No additional redirection in the Input/Output (I/O) path [0033] No additional write amplification. [0034] Integration with existing storage operating systems (e.g., the ONTAP data management software available from NetApp, Inc. of San Jose, CA). [0035] Distributed disk operations [0036] The ability to use all disks associated with a distributed storage system in a more uniform manner

[0037] In various examples described herein, disk space may be used more fluidly across all the individual storage systems (e.g., nodes) of a distributed storage system (e.g., a cluster of nodes working together), thereby eliminating silos of storage; and processing resource (e.g., central processing unit (CPU)) load may be distributed across the cluster. The proposed architecture seeks to prevent a given disk from being tied to any single node of the cluster by introducing a new construct referred to herein as a “dynamically extensible file system,” examples of which are described further below with reference to FIG. 6. In contrast to the entirety of a given storage device (e.g., a disk) being owned by a node-level aggregate and the aggregate file system being visible from only one node of a cluster as shown and described with reference to FIG. 5, the use of dynamically extensible file systems facilitates visibility by all nodes in the cluster to the entirety of a global physical volume block number (PVBN) space of the disks associated with a storage pool (e.g., a single “storage pod” (another new construct introduced herein) may be shared by all of the nodes of the cluster. According to one embodiment, the single storage pod implements a disaggregated storage space, thereby allowing the storage space addressed by the global PVBN space to be used on demand.

[0038] In some use cases, a node may become a bottleneck for processing power (e.g., CPU power) due to the way data is distributed across the nodes of the storage cluster. To remove the bottleneck, it may be desirable to be able to redistribute the data across the nodes within the same storage cluster (e.g., by moving a volume from one node to another) so as to provide a load balancing mechanism to redistribute the CPU load on the node. At present, distributed storage systems, for example, those implementing node-level aggregates, typically require data copy operations to be performed to accomplish such data redistribution. As those skilled in the art will appreciate, however, the performance of data copying may have the effect of creating even more load on an already overloaded node (at least during the performance of the data copying), which may potentially result in failure of the node. In order to mitigate such adverse impact on a node, embodiments herein propose an approach for performing a zero-copy volume move in which volume data may be maintained in place within a storage pod and need not be copied to move a given volume between nodes of the distributed storage system. For example, as described further below, only metadata (e.g., the block content of an index node (inode) of a container file representing the given volume) and desired property information (e.g., volume property metafiles) associated with the given volume may be copied from the source node to the destination node. Depending on the particular implementation, volume properties may include information related to, among other things, a fingerprint database containing fingerprints used for deduplication within the

volume, Storage Area Network (SAN) files, and files subject to compliance protections. At least one advantage of the proposed zero-copy volume move approach is that it allows traditional data copying (in which the time to complete the operation is proportional to (dependent on) the amount of data stored within the volume at issue) to be replaced with a constant-time operation that is independent of the amount of data stored within the volume at issue.

[0039] In one embodiment, each node of a cluster has access to do read and write to all the disks in a storage pod associated with the cluster. Given all the nodes have access to the same disks, a RAID subsystem or layer can now assimilate the same RAID tree from the same set of disks and present the global PVBN space to the file system (e.g., a write anywhere file system, such as the write anywhere file layout (WAFL) file system available from NetApp, Inc. of San Jose, CA). Using the global PVBN space, each node of the cluster can create an independent file system that it needs. As those skilled in the art will appreciate, it would be dangerous for each node to allocate from the same global PVBN space independently and without limitation. As such, examples of the proposed architecture restrict each dynamically extensible file system to use (consume) space only from the blocks assigned to it. As such, when performing writes, each dynamically extensible file system stays in its own lane without the need for complex access control mechanisms, such as locks.

[0040] As described further below, in some examples, the association of blocks to a dynamically extensible file system may be in large chunks of one or more gigabytes (GB), which are referred to herein as “allocation areas” (AAs) that each include multiple RAID stripes. The use of large, multi-GB chunks, as the unit of space allocation/assignment to dynamically extensible file systems facilitates ease of management (e.g., by way of reducing the frequency of ownership transfers among dynamically extensible file systems) of these AAs. The assignment of AAs to individual dynamically extensible file systems, which in turn are owned by nodes, additionally helps each node do its write allocation independently since, by definition an entire RAID stripe is owned by a single dynamically extensible file system. In some embodiments, dynamically extensible file systems also minimize or at least significantly reduce the need for internode communications. For example, dynamically extensible file systems can limit their coordination across nodes to situations in which space balancing is to be performed (e.g., responsive to a node running low on free storage space relative to the other nodes), which is not a frequent operation. Responsive to a space balancing trigger event, a given dynamically extensible file system (or the node owning given dynamically extensible file system on behalf of the given dynamically extensible file system) may request space be reassigned to it from one or more other dynamically extensible file systems. The combination of visibility into the entire global PVBN space and the use of dynamically extensible file systems and their association with a given portion of the disaggregated storage of a storage pod to which a given dynamically extensible file system has exclusive write access enables each node to run independently most of the time.

[0041] In the following description, numerous specific details are set forth in order to provide a thorough understanding of embodiments of the present disclosure. It will be apparent, however, to one skilled in the art that embodiments of the present disclosure may be practiced without some of these specific details. In other instances, well-known structures and devices are shown in block diagram form.

Terminology

[0042] Brief definitions of terms used throughout this application are given below.

[0043] The terms “connected” or “coupled” and related terms are used in an operational sense and are not necessarily limited to a direct connection or coupling. Thus, for example, two devices may be coupled directly, or via one or more intermediary media or devices. As another example, devices may be coupled in such a way that information can be passed there between, while not sharing any physical connection with one another. Based on the disclosure provided herein, one of ordinary skill in the art will appreciate a variety of ways in which connection or coupling exists in

accordance with the aforementioned definition.

[0044] If the specification states a component or feature “may”, “can”, “could”, or “might” be included or have a characteristic, that particular component or feature is not required to be included or have the characteristic.

[0045] As used in the description herein and throughout the claims that follow, the meaning of “a,” “an,” and “the” includes plural reference unless the context clearly dictates otherwise. Also, as used in the description herein, the meaning of “in” includes “in” and “on” unless the context clearly dictates otherwise.

[0046] The phrases “in an embodiment,” “according to one embodiment,” and the like generally mean the particular feature, structure, or characteristic following the phrase is included in at least one embodiment of the present disclosure and may be included in more than one embodiment of the present disclosure. Importantly, such phrases do not necessarily refer to the same embodiment.

[0047] As used herein a “cloud” or “cloud environment” broadly and generally refers to a platform through which cloud computing may be delivered via a public network (e.g., the Internet) and/or a private network. The National Institute of Standards and Technology (NIST) defines cloud computing as “a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.” P. Mell, T. Grance, The NIST Definition of Cloud Computing, National Institute of Standards and Technology, USA, 2011. The infrastructure of a cloud may be deployed in accordance with various deployment models, including private cloud, community cloud, public cloud, and hybrid cloud. In the private cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units), may be owned, managed, and operated by the organization, a third party, or some combination of them, and may exist on or off premises. In the community cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements, policy, and compliance considerations), may be owned, managed, and operated by one or more of the organizations in the community, a third party, or some combination of them, and may exist on or off premises. In the public cloud deployment model, the cloud infrastructure is provisioned for open use by the general public, may be owned, managed, and operated by a cloud provider or hyperscaler (e.g., a business, academic, or government organization, or some combination of them), and exists on the premises of the cloud provider. The cloud service provider may offer a cloud-based platform, infrastructure, application, or storage services as-a-service, in accordance with a number of service models, including Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS), and/or Infrastructure-as-a-Service (IaaS). In the hybrid cloud deployment model, the cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds).

[0048] As used herein, a “storage system” or “storage appliance” generally refers to a type of computing appliance or node, in virtual or physical form, that provides data to, or manages data for, other computing devices or clients (e.g., applications). The storage system may be part of a cluster of multiple nodes representing a distributed storage system. In various examples described herein, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider.

[0049] As used herein, the term “storage operating system” generally refers to computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a storage system (e.g., a node), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an

application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured for storage applications as described herein. In some embodiments, a light-weight data adaptor may be deployed on one or more server or compute nodes added to a cluster to allow compute-intensive data services to be performed without adversely impacting performance of storage operations being performed by other nodes of the cluster. The light-weight data adaptor may be created based on a storage operating system but, since the server node will not participate in handling storage operations on behalf of clients, the light-weight data adaptor may exclude various subsystems/modules that are used solely for serving storage requests and that are unnecessary for performance of data services. In this manner, compute intensive data services may be handled within the cluster by one of more dedicated compute nodes.

[0050] As used herein, a “cloud volume” generally refers to persistent storage that is accessible to a virtual storage system by virtue of the persistent storage being associated with a compute instance in which the virtual storage system is running. A cloud volume may represent a hard-disk drive (HDD) or a solid-state drive (SSD) from a pool of storage devices within a cloud environment that is connected to the compute instance through Ethernet or fibre channel (FC) switches as is the case for network-attached storage (NAS) or a storage area network (SAN). Non-limiting examples of cloud volumes include various types of SSD volumes (e.g., AWS Elastic Block Store (EBS) gp2, gp3, io1, and io2 volumes for EC2 instances) and various types of HDD volumes (e.g., AWS EBS st1 and sc1 volumes for EC2 instances).

[0051] As used herein a “consistency point” or “CP” generally refers to the act of writing data to disk and updating active file system pointers. In various examples, when a file system of a storage system receives a write request, it commits the data to permanent storage before the request is confirmed to the writer. Otherwise, if the storage system were to experience a failure with data only in volatile memory, that data would be lost, and underlying file structures could become corrupted. Physical storage appliances commonly use battery-backed high-speed non-volatile random access memory (NVRAM) as a journaling storage media to journal writes and accelerate write performance while providing permanence, because writing to memory is much faster than writing to storage (e.g., disk). Storage systems may also implement a buffer cache in the form of an in-memory cache to cache data that is read from data storage media (e.g., local mass storage devices or a storage array associated with the storage system) as well as data modified by write requests. In this manner, in the event a subsequent access relates to data residing within the buffer cache, the data can be served from local, high performance, low latency storage, thereby improving overall performance of the storage system. Virtual storage appliances may use NV storage backed by cloud volumes in place of NVRAM for journaling storage and for the buffer cache. Regardless of whether NVRAM or NV storage is utilized, the modified data may be periodically (e.g., every few seconds) flushed to the data storage media. As the buffer cache may be limited in size, an additional cache level may be provided by a victim cache, typically implemented within a slower memory or storage device than utilized by the buffer cache, that stores data evicted from the buffer cache. The event of saving the modified data to the mass storage devices may be referred to as a CP. At a CP, the file system may save any data that was modified by write requests to persistent data storage media. As will be appreciated, when using a buffer cache, there is a small risk of a system failure occurring between CPs, causing the loss of data modified after the last CP. Consequently, the storage system may maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP. This log may include a separate journal entry (e.g., including an operation header) for each storage request received from a client that results in a modification to the file system or data. Such entries for a given file may include, for example, “Create File,” “Write File Data,” and the like. Depending upon the operating mode or configuration of the storage system, each journal entry may also include the data to be written according to the corresponding request. The journal may be used in

the event of a failure to recover data that would otherwise be lost. For example, in the event of a failure, it may be possible to replay the journal to reconstruct the current state of stored data just prior to the failure. As described further below, in various examples there may be one or more predefined or configurable triggers (CP triggers). Responsive to a given CP trigger (or at a CP), the file system may save any data that was modified by write requests to persistent data storage media. [0052] As used herein, a “RAID stripe” generally refers to a set of blocks spread across multiple storage devices (e.g., disks of a disk array, disks of a disk shelf, or cloud volumes) to form a parity group (or RAID group).

[0053] As used herein, an “allocation area” or “AA” generally refers to a group of RAID stripes. In various examples described herein a single storage pod may be shared by a distributed storage system by assigning ownership of AAs to respective dynamically extensible file systems of a storage system.

[0054] As used herein, a “free allocation area” or “free AA” generally refers to an AA in which no PVBNs of the AA are marked as used, for example, by any active maps of a given dynamically extensible file system.

[0055] As used herein, a “partial allocation area” or “partial AA” generally refers to an AA in which one or more PVBNs of the AA are marked as in use (containing valid data), for example, by an active map of a given dynamically extensible file system. As discussed further below, in connection with space balancing, while it is preferable to perform AA ownership changes of free AAs, in various examples, space balancing may involve one dynamically extensible file system donating one or more partial AAs to another dynamically extensible file system. In such cases, the additional cost of copying portions of one or more associated data structures (e.g., bit maps, such as an active map, a refcount map, a summary map, an AA information map, and a space map) relating to storage space information may be incurred. No such additional cost is incurred when moving or changing ownership of free AAs. These associated data structures may, among other things, track which PVBNs are in use, track PVBN counts per AA (e.g., total used blocks and shared references to blocks) and other flags.

[0056] As used herein, a “storage pod” generally refers to a group of disks containing multiple RAID groups that are accessible from all storage systems (nodes) of a distributed storage system (cluster).

[0057] As used herein, a “data pod” generally refers to a set of storage systems (nodes) that share the same storage pod. In some examples, a data pod refers to a single cluster of nodes representing a distributed storage system. In other examples, there can be multiple data pods in a cluster. Data pods may be used to limit the fault domain and there can be multiple HA pairs of nodes within a data pod.

[0058] As used herein, an “active map” is a data structure that contains information indicative of which PVBNs of a distributed file system are in use. In one embodiment, the active map is represented in the form of a sparse bit map in which each PVBN of a global PVBN space of a storage pod has a corresponding Boolean value (or truth value) represented as a single bit, for example, in which the true (1) indicates the corresponding PVBN is in use and false (0) indicates the corresponding PVBN is not in use.

[0059] As used herein, a “dynamically extensible file system” or a “DEFS” generally refers to a file system of a data pod or a cluster that has visibility into the entire global PVBN space of a storage pod and hosts multiple volumes. A DEFS may be thought of as a data container or a storage container (which may be referred to as a storage segment container) to which AAs are assigned, thereby resulting in a more flexible and enhanced version of a node-level aggregate. As described further herein (for example, in connection with automatic space balancing), the storage space associated with one or more AAs of a given DEFS may be dynamically transferred or moved on demand to any other DEFS in the cluster by changing the ownership of the one or more AAs and moving associated AA tracking data structures as appropriate. This provides the unique ability to

independently scale each DEFS of a cluster. For example, DEFSs can shrink or grow dynamically over time to meet their respective storage needs and silos of storage space are avoided. In one embodiment, a distributed file system comprises multiple instances of the WAFL Copy-on-Write file system running on respective storage systems (nodes) of a distributed storage system (cluster) that represents the data pod. In various examples described herein, a given storage system (node) of a distributed storage system (cluster) may own one or more DEFSs including, for example, a log DEFS for hosting an operation log or journal of certain storage operations that have been performed by the node since the last CP and a data DEFS for hosting customer volumes or logical unit numbers (LUNs). As described further below, the partitioning/division of a storage pod into AAs (creation of a disaggregated storage space) and the distribution of ownership of AAs among DEFSs of multiple nodes of a cluster may facilitate implementation of a distributed storage system having a disaggregated storage architecture. In various examples described herein, each storage system may have its own portion of disaggregated storage to which it has the exclusive ability to perform write access, thereby simplifying storage management by, among other things, not requiring implementation of access control mechanisms, for example, in the form of locks. At the same time, each storage system also has visibility into the entirety of a global PVBN space, thereby allowing read access by a given storage system to any portion of the disaggregated storage regardless of which node of the cluster is the current owner of the underlying allocation areas. Based on disclosure provided herein, those skilled in the art will understand there are at least two types of disaggregation represented/achieved within various examples, including (i) the disaggregation of storage space provided by a storage pod by dividing or partitioning the storage space into AAs the ownership of which can be fluidly changed from one DEFS to another on demand and (ii) the disaggregation of the storage architecture into independent components, including the decoupling of processing resources and storage resources, thereby allowing them to be independently scaled. In one embodiment, the former (which may also be referred to as modular storage, partitioned storage, adaptable storage, or fluid storage) facilitates the latter.

[0060] As used herein, an “allocation area map” or “AA map” generally refers to a per dynamically extensible file system data structure or file (e.g., a metafile) that contains information at an AA-level of granularity indicative of which AAs are assigned to or “owned” by a given dynamically extensible file system.

[0061] A “node-level aggregate” generally refers to a file system of a single storage system (node) that holds multiple volumes created over one or more RAID groups, in which the node owns the entire PVBN space of the collection of disks of the one or more RAID groups. Node-level aggregates are only accessible from a single storage system (node) of a distributed storage system (cluster) at a time.

[0062] As used herein, an “inode” generally refers to a file data structure maintained by a file system that stores metadata for container files (e.g., volumes, directories, subdirectories, disk files, etc.). An inode may include, among other things, location, file size, permissions needed to access a given container file with which it is associated as well as creation, read, and write timestamps, and one or more flags.

[0063] As used herein, a “storage volume” or “volume” generally refers to a container in which applications, databases, and file systems store data. A volume is a logical component created for the host to access storage on a storage array. A volume may be created from the capacity available in storage pod, a pool, or a volume group. A volume has a defined capacity. Although a volume might consist of more than one drive, a volume appears as one logical component to the host. Non-limiting examples of a volume include a flexible volume and a flexgroup volume.

[0064] As used herein, a “flexible volume” generally refers to a type of storage volume that may be efficiently distributed across multiple storage devices. A flexible volume may be capable of being resized to meet changing business or application requirements. In some embodiments, a storage system may provide one or more aggregates and one or more storage volumes distributed across a

plurality of nodes interconnected as a cluster. Each of the storage volumes may be configured to store data such as files and logical units. As such, in some embodiments, a flexible volume may be comprised within a storage aggregate and further comprises at least one storage device. The storage aggregate may be abstracted over a RAID plex where each plex comprises a RAID group. Moreover, each RAID group may comprise a plurality of storage disks. As such, a flexible volume may comprise data storage spread over multiple storage disks or devices. A flexible volume may be loosely coupled to its containing aggregate. A flexible volume can share its containing aggregate with other flexible volumes. Thus, a single aggregate can be the shared source of all the storage used by all the flexible volumes contained by that aggregate. A non-limiting example of a flexible volume is a NetApp ONTAP Flex Vol volume.

[0065] As used herein, a “flexgroup volume” generally refers to a single namespace that is made up of multiple constituent/member volumes. A non-limiting example of a flexgroup volume is a NetApp ONTAP FlexGroup volume that can be managed by storage administrators, and which acts like a NetApp Flex Vol volume. In the context of a flexgroup volume, “constituent volume” and “member volume” are interchangeable terms that refer to the underlying volumes (e.g., flexible volumes) that make up the flexgroup volume.

Example Distributed Storage System Cluster

[0066] FIG. 1 is a block diagram illustrating a plurality of nodes **110a-b** interconnected as a cluster **100** in accordance with an embodiment of the present disclosure. In the context of the present example, the nodes **110a-b** comprise various functional components that cooperate to provide a distributed storage system architecture of the cluster **100**. To that end, in the context of the present example, each node is generally organized as a network element (e.g., network element **120a** or **120b**) and a disk element (e.g., disk element **150a** or **150b**). The network element includes functionality that enables the node to connect to clients (e.g., client **180**) over a computer network **140**, while each disk element **350** connects to one or more storage devices, such as disks, of one or more disk arrays (not shown) or of one or more storage shelves (not shown), represented as a single shared storage pod **145**.

[0067] In the context of the present example, the nodes **110a-b** are interconnected by a cluster switching fabric **151** which, in an example, may be embodied as a Gigabit Ethernet switch. It should be noted that while there is shown an equal number of network and disk elements in the illustrative cluster **100**, there may be differing numbers of network and/or disk elements. For example, there may be a plurality of network elements and/or disk elements interconnected in a cluster configuration **100** that does not reflect a one-to-one correspondence between the network and disk elements. As such, the description of a node comprising one network element and one disk element should be taken as illustrative only.

[0068] Clients may be general-purpose computers configured to interact with the node in accordance with a client/server model of information delivery. That is, each client (e.g., client **180**) may request the services of the node, and the node may return the results of the services requested by the client, by exchanging packets over the network **140**. The client may issue packets including file-based access protocols, such as the Common Internet File System (CIFS) protocol or Network File System (NFS) protocol, over the Transmission Control Protocol/Internet Protocol (TCP/IP) when accessing information in the form of files and directories. Alternatively, the client may issue packets including block-based access protocols, such as the Small Computer Systems Interface (SCSI) protocol encapsulated over TCP (iSCSI) and SCSI encapsulated over Fibre Channel (FCP), when accessing information in the form of blocks. In various examples described herein, an administrative user (not shown) of the client may make use of a user interface (UI) presented by the cluster or a command line interface (CLI) of the cluster to, among other things, establish a data protection relationship between a source volume and a destination volume (e.g., a mirroring relationship specifying one or more policies associated with creation, retention, and transfer of snapshots), defining snapshot and/or backup policies, and association of snapshot policies with

snapshots.

[0069] Disk elements **150a** and **150b** are illustratively connected to disks (not shown) within that may be organized into disk arrays within the storage pod **145**. Alternatively, storage devices other than disks may be utilized, e.g., flash memory, optical storage, solid state devices, etc. As such, the description of disks should be taken as exemplary only.

[0070] In general, various embodiments envision a cluster (e.g., cluster **100**) in which every node (e.g., nodes **110a-b**) can essentially talk to every storage device (e.g., disk) in the storage pod **145**. This is in contrast to the distributed storage system architecture described with reference to FIG. 5. In examples described herein, all nodes (e.g., nodes **110a-b**) of the cluster have visibility and read access to an entirety of a global PVBN space of the storage pod **145**, for example, via an interconnect layer **142**. As described further below, according to one embodiment, the storage within the storage pod **145** is grouped into distinct allocation areas (AAs) than can be assigned to a given dynamically extensible file system (DEFS) of a node to facilitate implementation disaggregated storage. In examples described herein, the AAs assigned to a given DEFS may be said to “own” the assigned AAs and the node owning the given DEFS has the exclusive write access to the associated PVBNs and the exclusive ability to perform write allocation from such blocks. In one embodiment, each node has its own view of a portion of the disaggregated storage represented by the assignment of, for example, via respective allocation area (AA) maps and active maps. This granular assignment of AAs and ability to fluidly change ownership of AAs as needed facilitates the elimination of per-node storage silos and provides higher and more predictable performance, which further translate into improved storage utilization and improvements in cost effectiveness of the storage solution.

[0071] Depending on the particular implementation, the interconnect layer **142** may be represented by an intermediate switching topology or some other interconnectivity layer or disk switching layer between the disks in the storage pod **145** and the nodes. Non-limiting examples of the interconnect layer **150** include one or more fiber channel switches or one or more non-volatile memory express (NVMe) fabric switches. Additional details regarding the storage pod **145**, DEFSs, AA maps, active maps, and the use, ownership, and sharing (transferring of ownership) of AAs are described further below.

Example Storage System Node

[0072] FIG. 2 is a block diagram of a node **200** that is illustratively embodied as a storage system comprising a plurality of processors (e.g., processors **222a-b**), a memory **224**, a network adapter **225**, a cluster access adapter **226**, a storage adapter **228** and local storage **230** interconnected by a system bus **223**. Node **200** may be analogous to nodes **110a** and **110b** of FIG. 1. The local storage **230** comprises one or more storage devices, such as disks, utilized by the node to locally store configuration information (e.g., in configuration table **235**). The cluster access adapter **226** comprises a plurality of ports adapted to couple the node **200** to other nodes of the cluster (e.g., cluster **100**). Illustratively, Ethernet is used as the clustering protocol and interconnect media, although it will be apparent to those skilled in the art that other types of protocols and interconnects may be utilized within the cluster architecture described herein. Alternatively, where the network elements and disk elements are implemented on separate storage systems or computers, the cluster access adapter **226** is utilized by the network and disk element for communicating with other network and disk elements in the cluster.

[0073] In the context of the present example, each node **200** is illustratively embodied as a dual processor storage system executing a storage operating system **210** that implements a high-level module, such as a file system, to logically organize the information as a hierarchical structure of named directories, files and special types of files called virtual disks (hereinafter generally “blocks”) on the disks. However, it will be apparent to those of ordinary skill in the art that the node **200** may alternatively comprise a single or more than two processor system. Illustratively, one processor (e.g., processor **222a**) may execute the functions of the network element (e.g., network

element **120a** or **120b**) on the node, while the other processor (e.g., processor **222b**) may execute the functions of the disk element (e.g., disk element **150a** or **150b**).

[0074] The memory **224** illustratively comprises storage locations that are addressable by the processors and adapters for storing software program code and data structures associated with the subject matter of the disclosure. The processor and adapters may, in turn, comprise processing elements and/or logic circuitry configured to execute the software code and manipulate the data structures. The storage operating system **210**, portions of which is typically resident in memory and executed by the processing elements, functionally organizes the node **200** by, inter alia, invoking storage operations in support of the storage service implemented by the node. It will be apparent to those skilled in the art that other processing and memory means, including various computer readable media, may be used for storing and executing program instructions pertaining to the disclosure described herein.

[0075] The network adapter **225** comprises a plurality of ports adapted to couple the node **200** to one or more clients (e.g., client **180**) over point-to-point links, wide area networks, virtual private networks implemented over a public network (Internet) or a shared local area network. The network adapter **225** thus may comprise the mechanical, electrical and signaling circuitry needed to connect the node to a network (e.g., computer network **140**). Illustratively, the network may be embodied as an Ethernet network or a Fibre Channel (FC) network. Each client (e.g., client **180**) may communicate with the node over network by exchanging discrete frames or packets of data according to pre-defined protocols, such as TCP/IP.

[0076] The storage adapter **228** cooperates with the storage operating system **210** executing on the node **200** to access information requested by the clients. The information may be stored on any type of attached array of writable storage device media such as video tape, optical, DVD, magnetic tape, bubble memory, electronic random access memory, micro-electromechanical and any other similar media adapted to store information, including data and parity information. However, as illustratively described herein, the information is stored on disks (e.g., associated with storage pod **145**). The storage adapter comprises a plurality of ports having input/output (I/O) interface circuitry that couples to the disks over an I/O interconnect arrangement, such as a conventional high-performance, FC link topology.

[0077] Storage of information on each disk array may be implemented as one or more storage “volumes” that comprise a collection of physical storage disks or cloud volumes cooperating to define an overall logical arrangement of volume block number (VBN) space on the volume(s). Each logical volume is generally, although not necessarily, associated with its own file system. The disks within a logical volume/file system are typically organized as one or more groups, wherein each group may be operated as a Redundant Array of Independent (or Inexpensive) Disks (RAID). Most RAID implementations, such as a RAID-4 level implementation, enhance the reliability/integrity of data storage through the redundant writing of data “stripes” across a given number of physical disks in the RAID group, and the appropriate storing of parity information with respect to the striped data. An illustrative example of a RAID implementation is a RAID-4 level implementation, although it should be understood that other types and levels of RAID implementations may be used in accordance with the inventive principles described herein.

[0078] While in the context of the present example, the node may be a physical host, it is to be appreciated the node may be implemented in virtual form. For example, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider. As such, a cluster representing a distributed storage system may be comprised of multiple physical nodes (e.g., node **200**) or multiple virtual nodes (virtual storage systems).

Example Storage Operating System

[0079] To facilitate access to the disks (e.g., disks within one or more disk arrays of a storage pod, such as storage pod **145** of FIG. **1**), a storage operating system (e.g., storage operating system **300**, which may be analogous to storage operating system **210**) may implement a write-anywhere file

system that cooperates with one or more virtualization modules to “virtualize” the storage space provided by disks. The file system logically organizes the information as a hierarchical structure of named directories and files on the disks. Each “on-disk” file may be implemented as set of disk blocks configured to store information, such as data, whereas the directory may be implemented as a specially formatted file in which names and links to other files and directories are stored. The virtualization module(s) allow the file system to further logically organize information as a hierarchical structure of blocks on the disks that are exported as named logical unit numbers (LUNs).

[0080] Illustratively, the storage operating system may be the Data ONTAP operating system available from NetApp, Inc., San Jose, Calif. that implements the WAFL file system. However, it is expressly contemplated that any appropriate storage operating system may be enhanced for use in accordance with the inventive principles described herein. As such, where the term “WAFL” is employed, it should be taken broadly to refer to any file system (e.g., a copy-on-write file system, a write-anywhere file system, or the like) that is otherwise adaptable to the teachings of this disclosure.

[0081] FIG. 3 is a block diagram illustrating a storage operating system **300** in accordance with an embodiment of the present disclosure. In the context of the present example, the storage operating system **300** is shown including a series of software layers organized to form an integrated network protocol stack or, more generally, a multi-protocol engine **325** that provides data paths for clients to access information stored on the node using block and file access protocols. The multi-protocol engine includes a media access layer **312** of network drivers (e.g., gigabit Ethernet drivers) that interfaces to network protocol layers, such as the IP layer **314** and its supporting transport mechanisms, the TCP layer **316** and the User Datagram Protocol (UDP) layer **315**. A file system protocol layer provides multi-protocol file access and, to that end, includes support for the Direct Access File System (DAFS) protocol **318**, the NFS protocol **320**, the CIFS protocol **322** and the Hypertext Transfer Protocol (HTTP) protocol **324**. A VI layer **326** implements the VI architecture to provide direct access transport (DAT) capabilities, such as ROMA, as required by the DAFS protocol **318**. An iSCSI driver layer **328** provides block protocol access over the TCP/IP network protocol layers, while a FC driver layer **330** receives and transmits block access requests and responses to and from the node. The FC and iSCSI drivers provide FC-specific and iSCSI-specific access control to the blocks and, thus, manage exports of LUNs to either iSCSI or FCP or, alternatively, to both iSCSI and FCP when accessing the blocks on the node (e.g., node **200**).

[0082] In addition, the storage operating system may include a series of software layers organized to form a storage server **365** that provides data paths for accessing information stored on storage devices (e.g., disks that are part of a shared storage pod, such as storage pod **145**) accessible by the node. To that end, the storage server **365** includes a file system module **360** in cooperating relation with a remote access module **370**, a RAID system module **380** and a disk driver system module **390**. The RAID system **380** manages the storage and retrieval of information to and from the volumes/disks in accordance with I/O operations, while the disk driver system **390** implements a disk access protocol such as, e.g., the SCSI protocol.

[0083] The file system **360** may implement a virtualization system of the storage operating system **300** through the interaction with one or more virtualization modules illustratively embodied as, for example, a virtual disk (vdisk) module (not shown) and a SCSI target module **335**. The SCSI target module **335** is generally disposed between the FC and iSCSI drivers **328**, **330** and the file system **360** to provide a translation layer of the virtualization system between the block (LUN) space and the file system space, where LUNs are represented as blocks.

[0084] The file system **360** is illustratively a message-based system that provides logical volume management capabilities for use in access to the information stored on the storage devices, such as disks. That is, in addition to providing file system semantics, the file system **360** provides functions normally associated with a volume manager. These functions include (i) aggregation of the disks,

(ii) aggregation of storage bandwidth of the disks, and (iii) reliability guarantees, such as mirroring and/or parity (RAID). The file system **360** illustratively implements an exemplary a file system having an on-disk format representation that is block-based using, e.g., 4 kilobyte (KB) blocks and using index nodes (“inodes”) to identify files and file attributes (such as creation time, access permissions, size and block location). The file system uses files to store meta-data describing the layout of its file system; these meta-data files include, among others, an inode file. A file handle, i.e., an identifier that includes an inode number, is used to retrieve an inode from disk.

[0085] Broadly stated, all inodes of the write-anywhere file system are organized into the inode file. A file system (fs) info block specifies the layout of information in the file system and includes an inode of a file that includes all other inodes of the file system. Each logical volume (file system) has an fsinfo block that is preferably stored at a fixed location within, e.g., a RAID group. The inode of the inode file may directly reference (point to) data blocks of the inode file or may reference indirect blocks of the inode file that, in turn, reference data blocks of the inode file. Within each data block of the inode file are embedded inodes, each of which may reference indirect blocks that, in turn, reference data blocks of a file.

[0086] Operationally, a request from a client (e.g., client **180**) is forwarded as a packet over a computer network (e.g., computer network **140**) and onto a node (e.g., node **200**) where it is received at a network adapter (e.g., network adaptor **225**). A network driver (of layer **312** or layer **330**) processes the packet and, if appropriate, passes it on to a network protocol and file access layer for additional processing prior to forwarding to the write-anywhere file system **360**. Here, the file system generates operations to load (retrieve) the requested data from persistent storage (e.g., a disk that is part of a shared storage pod, such as storage pod **145**) accessible by the node if the requested data is not resident “in core”, i.e., in memory **224**. If the information is not in memory, the file system **360** indexes into the inode file using the inode number to access an appropriate entry and retrieve a logical VBN. The file system then passes a message structure including the logical VBN to the RAID system **380**; the logical VBN is mapped to a disk identifier and disk block number (disk,dbn) and sent to an appropriate driver (e.g., SCSI) of the disk driver system **390**. The disk driver accesses the dbn from the specified disk and loads the requested data block(s) in memory for processing by the node. Upon completion of the request, the node (and operating system) returns a reply to the client **180** over the network **140**.

[0087] The remote access module **370** is operatively interfaced between the file system module **360** and the RAID system module **380**. Remote access module **370** is illustratively configured as part of the file system to implement the functionality to determine whether a newly created container file, such as volume, a directory, a subdirectory or file, should be stored locally or remotely.

Alternatively, the remote access module **370** may be separate from the file system. As such, the description of the remote access module being part of the file system should be taken as exemplary only. Further, the remote access module **370** determines which remote flexible volume should store a new subdirectory if a determination is made that the subdirectory is to be stored remotely. More generally, the remote access module **370** implements the heuristics algorithms used for the adaptive data placement. However, it should be noted that the use of a remote access module should be taken as illustrative. In alternative aspects, the functionality may be integrated into the file system or other module of the storage operating system. As such, the description of the remote access module **370** performing certain functions should be taken as exemplary only.

[0088] It should be noted that the software “path” through the storage operating system layers described above needed to perform data storage access for the client request received at the node may alternatively be implemented in hardware. That is, a storage access request data path may be implemented as logic circuitry embodied within a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC). This type of hardware implementation increases the performance of the storage service provided by node **200** in response to a request issued by client **180**. Alternatively, the processing elements of adapters **225**, **228** may be configured to offload some

or all of the packet processing and storage access operations, respectively, from processor **222**, to thereby increase the performance of the storage service provided by the node. It is expressly contemplated that the various processes, architectures and procedures described herein can be implemented in hardware, firmware or software.

[0089] As used herein, the term “storage operating system” generally refers to the computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a node (e.g., node **200**), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured for storage applications as described herein.

[0090] In addition, it will be understood to those skilled in the art that aspects of the disclosure described herein may apply to any type of special-purpose (e.g., file server, filer or storage serving appliance) or general-purpose computer, including a standalone computer or portion thereof, embodied as or including a storage system. Moreover, the teachings contained herein can be adapted to a variety of storage system architectures including, but not limited to, a network-attached storage environment, a storage area network and disk assembly directly attached to a client or host computer. The term “storage system” should therefore be taken broadly to include such arrangements in addition to any subsystems configured to perform a storage function and associated with other equipment or systems. It should be noted that while this description is written in terms of a write anywhere file system, the teachings of the subject matter may be utilized with any suitable file system, including a write in place file system.

Example Cluster Fabric (CF) Protocol

[0091] Illustratively, the storage server **365** is embodied as disk element (or disk blade **350**, which may be analogous to disk element **150a** or **150b**) of the storage operating system **300** to service one or more volumes of array **160**. In addition, the multi-protocol engine **325** is embodied as network element (or network blade **310**, which may be analogous to network element **120a** or **120b**) to (i) perform protocol termination with respect to a client issuing incoming data access request packets over the network (e.g., network **140**), as well as (ii) redirect those data access requests to any storage server **365** of the cluster (e.g., cluster **100**). Moreover, the network element **310** and disk element **350** cooperate to provide a highly scalable, distributed storage system architecture of the cluster. To that end, each module may include a cluster fabric (CF) interface module (e.g., CF interface **340a** and **340b**) adapted to implement intra-cluster communication among the nodes (e.g., node **110a** and **110b**). In the context of a distributed storage architecture as described below with reference to FIG. 5 in which node-level aggregates are employed, the CF protocol facilitates, among other things, internode communications relating to data access requests. It is to be appreciated such internode communications relating to data access requests are not needed in the context of a distributed storage architecture as described below with reference to FIG. 6 in which each node of a cluster has visibility and access to the entirety of a global PVBN space of a storage pod (via their respective DEFSs). However, in various embodiments, some limited amount of internode communications, for example, relating to storage space reporting (or simply space reporting) and storage space requests (e.g., requests for donations of AAs) continue to be useful. As described further below, such internode communications may make use of the CF protocol or other forms of internode communications, including message passing via on-wire communications and/or the use of one or more persistent message queues (or on-disk message queues), which may make use of the fact that all nodes can read from all disk of a storage pod. For example, a persistent message queue may be maintained at the node and/or DEFS-level of granularity in which each node and/or DEFS has a message queue to which others can post messages destined for the node or DEFS (as the case may be). In one embodiment, each DEFS has an associated inbound queue on which it receives messages sent by another DEFS in the cluster and an associated outbound queue

on which it posts messages intended for delivery to another DEFS in the cluster

[0092] The protocol layers, e.g., the NFS/CIFS layers and the iSCSI/IFC layers, of the network element **310** may function as protocol servers that translate file-based and block based data access requests from clients into CF protocol messages used for communication with the disk element **350**. That is, the network element servers may convert the incoming data access requests into file system primitive operations (commands) that are embedded within CF messages by the CF interface module **340** for transmission to the disk elements of the cluster.

[0093] Further, in an illustrative aspect of the disclosure, the network element and disk element are implemented as separately scheduled processes of storage operating system **300**; however, in an alternate aspect, the modules may be implemented as pieces of code within a single operating system process. Communication between a network element and disk element may thus illustratively be effected through the use of message passing between the modules although, in the case of remote communication between a network element and disk element of different nodes, such message passing occurs over a cluster switching fabric (e.g., cluster switching fabric **151**). A known message-passing mechanism provided by the storage operating system to transfer information between modules (processes) is the Inter Process Communication (IPC) mechanism. The protocol used with the IPC mechanism is illustratively a generic file and/or block-based “agnostic” CF protocol that comprises a collection of methods/functions constituting a CF application programming interface (API). Examples of such an agnostic protocol are the SpinFS and SpinNP protocols available from NetApp, Inc.

[0094] The CF interface module **340** implements the CF protocol for communicating file system commands among the nodes or modules of cluster. Communication may be illustratively effected by the disk element exposing the CF API to which a network element (or another disk element) issues calls. To that end, the CF interface module **340** may be organized as a CF encoder and CF decoder. The CF encoder of, e.g., CF interface **340a** on network element **310** encapsulates a CF message as (i) a local procedure call (LPC) when communicating a file system command to a disk element **350** residing on the same node **200** or (ii) a remote procedure call (RPC) when communicating the command to a disk element residing on a remote node of the cluster **100**. In either case, the CF decoder of CF interface **340b** on disk element **350** de-encapsulates the CF message and processes the file system command.

[0095] Illustratively, the remote access module **370** may utilize CF messages to communicate with remote nodes to collect information relating to remote flexible volumes. A CF message is used for RPC communication over the switching fabric between remote modules of the cluster; however, it should be understood that the term “CF message” may be used generally to refer to LPC and RPC communication between modules of the cluster. The CF message includes a media access layer, an IP layer, a UDP layer, a reliable connection (RC) layer and a CF protocol layer. The CF protocol is a generic file system protocol that may convey file system commands related to operations contained within client requests to access data containers or container files represented within or stored on the cluster; the CF protocol layer is that portion of a message that carries the file system commands. Illustratively, the CF protocol is datagram based and, as such, involves transmission of messages or “envelopes” in a reliable manner from a source (e.g., a network element **310**) to a destination (e.g., a disk element **350**). The RC layer implements a reliable transport protocol that is adapted to process such envelopes in accordance with a connectionless protocol, such as UDP.

Example File System Layout

[0096] In one embodiment, a data container or container file is represented in the write-anywhere file system as an inode data structure adapted for storage on the disks of a storage pod (e.g., storage pod **145**). The inode may include a meta-data section and a data section. The information stored in the meta-data section of each inode describes the container file (e.g., a file, a snapshot, etc.) and, as such, may include the type (e.g., regular, directory, vdisk) of file, its size, time stamps (e.g., access and/or modification time) and ownership (e.g., user identifier (UID) and group ID (GID), of the

file, and a generation number. The contents of the data section of each inode may be interpreted differently depending upon the type of file (inode) defined within the type field. For example, the data section of a directory inode includes meta-data controlled by the file system, whereas the data section of a regular inode includes file system data. In this latter case, the data section includes a representation of the data associated with the file.

[0097] Specifically, the data section of a regular on-disk inode (or disk inode) may include file system data or pointers, the latter referencing 4 KB data blocks on disk used to store the file system data. Each pointer may be a block number (e.g., a logical VBN to facilitate efficiency among the file system and the RAID system when accessing the data on disks). Given the restricted size (e.g., 128 bytes) of the inode, file system data having a size that is less than or equal to 64 bytes is represented, in its entirety, within the data section of that inode. However, if the length of the contents of the data container or container file exceeds 64 bytes but less than or equal to 64 KB, then the data section of the inode (e.g., a first level inode) comprises up to 16 pointers, each of which references a 4 KB block of data on the disk.

[0098] Moreover, if the size of the data is greater than 64 KB but less than or equal to 64 megabytes (MB), then each pointer in the data section of the inode (e.g., a second level inode) references an indirect block (e.g., a first level L1 block) that contains 224 pointers, each of which references a 4 KB data block on disk. For file system data having a size greater than 64 MB, each pointer in the data section of the inode (e.g., a third level L3 inode) references a double-indirect block (e.g., a second level L2 block) that contains 224 pointers, each referencing an indirect (e.g., a first level L1) block. The indirect block, in turn, which contains 224 pointers, each of which references a 4 kB data block on disk. When accessing a file, each block of the file may be loaded from disk into memory (e.g., memory **224**). In other embodiments, higher levels are also possible that may be used to handle larger data container or container file sizes.

[0099] When an on-disk inode (or block) is loaded from disk into memory, its corresponding in-core structure embeds the on-disk structure. The in-core structure is a block of memory that stores the on-disk structure plus additional information needed to manage data in the memory (but not on disk). The additional information may include, e.g., a “dirty” bit. After data in the inode (or block) is updated/modified as instructed by, e.g., a write operation, the modified data is marked “dirty” using the dirty bit so that the inode (block) can be subsequently “flushed” (stored) to disk.

[0100] According to one embodiment, a file in a file system comprises a buffer tree that provides an internal representation of blocks for a file loaded into memory and maintained by the write-anywhere file system **360**. A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. In other embodiments, there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (e.g., and inode) includes pointers that ultimately reference data blocks used to store the actual data of the file. That is, the data of file are contained in data blocks and the locations of these blocks are stored in the indirect blocks of the file. Each level 1 indirect block may include pointers to as many as 224 data blocks. According to the “write anywhere” nature of the file system, these blocks may be located anywhere on the disks.

[0101] In one embodiment, a file system layout is provided that apports an underlying physical volume into one or more virtual volumes (or flexible volumes) of a storage system, such as node **200**. Depending on the particular implementation, the underlying physical volume may be a DEFS or an aggregate comprising one or more groups of disks, such as RAID groups. When using a storage pod (e.g., storage pod **145**) all DEFSs may share a common global PVBN space. In other examples, the aggregate may have its own physical volume block number (PVBN) space. The DEFS or aggregate, as the case may be, also maintains meta-data, such as block allocation structures, within that PVBN space. Each flexible volume has its own virtual volume block number (VVBN) space and maintains meta-data, such as block allocation structures, within that VVBN space. Each flexible volume is a file system that is associated with a container file; the container

file is a file in the DEFS or aggregate that contains all blocks used by the flexible volume.

Moreover, each flexible volume comprises data blocks and indirect blocks that contain block pointers that point at either other indirect blocks or data blocks.

[0102] In a further embodiment, PVBNs are used as block pointers within buffer trees of files stored in a flexible volume. This “hybrid” flexible volume example involves the insertion of only the PVBN in the parent indirect block (e.g., inode or indirect block). On a read path of a logical volume, a “logical” volume (vol) info block has one or more pointers that reference one or more fsinfo blocks, each of which, in turn, points to an inode file and its corresponding inode buffer tree. The read path on a flexible volume is generally the same, following PVBNs (instead of VVBNs) to find appropriate locations of blocks; in this context, the read path (and corresponding read performance) of a flexible volume is substantially similar to that of a physical volume. Translation from PVBN-to-disk,dbn occurs at the file system/RAID system boundary of the storage operating system **300**.

[0103] In a dual VBN hybrid flexible volume example, both a PVBN and its corresponding VVBN are inserted in the parent indirect blocks in the buffer tree of a file. That is, the PVBN and VVBN are stored as a pair for each block pointer in most buffer tree structures that have pointers to other blocks, e.g., level 1 (L1) indirect blocks, inode file level 0 (L0) blocks.

[0104] A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. Note that there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (and inode) include PVBN/VVBN pointer pair structures that ultimately reference data blocks used to store the actual data of the file. The PVBNs reference locations on disks of the aggregate, whereas the VVBNs reference locations within files of the flexible volume. The use of PVBNs as block pointers in the indirect blocks provides efficiencies in the read paths, while the use of VVBN block pointers provides efficient access to required meta-data. That is, when freeing a block of a file, the parent indirect block in the file contains readily available VVBN block pointers, which avoids the latency associated with accessing an owner map to perform PVBN-to-VVBN translations; yet, on the read path, the PVBN is available.

Example Hierarchical Inode Tree

[0105] FIG. 4 is a block diagram illustrating a tree of blocks **400** representing a simplified view of an example a file system layout in accordance with an embodiment of the present disclosure. In one embodiment, the data storage system nodes (e.g., data storage systems **110a-b**) make use of a write anywhere file system (e.g., the WAFL file system). The write anywhere file system may represent a UNIX compatible file system that is optimized for network file access. In the context of the present example, the write anywhere file system is a block-based file system that represents file system data (e.g., a block map file and an inode map file), meta-data files, and data containers or container files (e.g., volumes, subdirectories, and regular files) in a tree of blocks (e.g., tree of blocks **400**). Keeping meta-data in files allows the file system to write meta-data blocks anywhere on disk and makes it easier to increase the size of the file system on the fly.

[0106] In this simplified example, the tree of blocks **400** has a root inode **410**, which describes an inode map file (not shown), made up of inode file indirect blocks **420** and inode file data blocks **430**. The file system may use inodes (e.g., inode file data blocks **430**) to describe container files (e.g., container file **441a** and container file **441b**). In one embodiment, each inode contains **16** block pointers (e.g., PVBNs specifying respective data block locations within the DEFS) to indicate which blocks (e.g., of 4 KB) belong to a given container file (e.g., a volume, a directory, a subdirectory, or a file). Inodes for container files smaller than 64 KB may use the 156 block pointers to point to file data blocks or simply data blocks (e.g., regular file data blocks, which may also be referred to herein as L0 blocks **450**). Inodes for files smaller than 64 MB may point to indirect blocks (e.g., regular file indirect blocks, which may also be referred to herein as L1 blocks **440**), which point to actual file data. Inodes for larger container files or data containers may point

to doubly indirect blocks. For very small files, data may be stored in the inode itself in place of the block pointers. Additional details regarding a specific implementation of a write anywhere file system are provided in U.S. Pat. No. 6,239,356, which is incorporated by reference herein in its entirety for all purposes.

[0107] As will be appreciated by those skilled in the art given the above-described file system layout, yet another advantage of DEFSs are their ability to facilitate storage space balancing and/or load balancing. This comes from the fact that the entire global PVBN space of a storage pod is visible to all DEFSs of the cluster and therefore any given DEFS can get access to an entire container file by copying the top-most PVBN from the inode on another tree.

[0108] Furthermore, as described herein, with disaggregated storage, DEFSs that are hosted on each node are able to see the entire storage space of a storage pod (e.g., storage pod **145**). For example, as described below with reference to FIGS. **13A-15**, this opens up to a new capability of moving a container file (e.g., a volume) in one DEFS to another by simply moving the disk inode content of the container file inode that hosts the volume to the destination DEFS. Since the entirety of the global PVBN space is visible on both nodes, moving the top-most PVBNs to the destination node may be used to allow the volume to be relocated to and mounted on the destination node.

Example of a Distributed Storage System Architecture with Storage Silos

[0109] FIG. **5** is a block diagram illustrating a distributed storage system architecture **500** in which the entirety of a given disk and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space. In the context of FIG. **5**, node **510a** and node **510b** may represent a two-node cluster in which the nodes are high-availability (HA) partners. For example, one node may represent a primary node and the other may represent a secondary node in which pairwise disk connectivity supports a pairwise failover model. As shown, each node includes respective active maps (e.g., active map **541a** and active map **541b**) and a sets of disks (in this case, ten disks) they can talk to. The nodes may partition the disks among themselves as aggregates (e.g., data aggregate **520a** and data aggregate **520b**) and at steady state both nodes will work on their own subset of disks representing a one or more RAID groups (in this case, four data disks and one parity disk, forming a single RAID group). A RAID layer or subsystem (not shown) of a storage operating system (not shown) of each node may present respective separate and independent PVBN spaces (e.g., PVBN space **540a** and PVBN space **540b**) to a file system layer (not shown) of the node.

[0110] In this example, therefore, data aggregate **520a** has visibility only to a first PVBN space (e.g., PVBN space **540a**) and data aggregate **520b** has visibility only to a second PVBN space (e.g., PVBN space **540b**). When data is stored to volume **530a** or **530b**, it is striped across the subset of disks that are part of data aggregate **520a**; and when data is stored to volume **530c** or **530d**, it is striped across the subset of disks that are part of data aggregate **520b**. Active map **541a** is a data structure (e.g., a bit map with one bit per PVBN) that identifies the PVBNs within PVBN space **540a** that are in use by data aggregate **520a**. Similarly, active map **541b** is a data structure (e.g., a bit map with one bit per PVBN) that identifies the PVBNs within PVBN space **540b** that are in use by data aggregate **520b**.

[0111] As can be seen, for any given disk, the entire disk is owned by a particular aggregate and the aggregate file system is only visible from one node. Similarly, for any given RAID group, the available storage space of the entire RAID group is useable only by a single node. There are various other disadvantages to the architecture shown in FIG. **5**. For example, moving a volume from one aggregate to another requires copying of data (e.g., reading all the blocks used by the volume and writing them to the new location), with an elaborate handover sequence between the aggregates involved. Additionally, there are scenarios in which one data aggregate may run out of storage space while the other still has plentiful free storage space, resulting in ineffective usage of the storage space provided by the disks. While the size of the PVBN space of an aggregate may be increased, doing so typically requires an administrative user to monitor the storage space on each

node-level aggregate and add one or more disks and/or RAID groups to the aggregate. As described further below with reference to FIG. 6, with DEFSs storage space is added to a common pool of storage referred to herein as a “storage pod” and space is available for consumption by any DEFS in the cluster, thereby making space management much simpler and facilitating the automatic balancing of storage space without administrator involvement.

Example Distributed System Architecture Providing Disaggregated Storage

[0112] Before getting into the details of a particular example, various properties, constructs, and principles relating to the use and implementation of DEFSs will now be discussed. As noted above, it is desirable to make the global PVBN space of the entire storage pool (e.g., storage pod 145) available on each DEFS of a data pod, which may include one or more clusters. This feature facilitates the performance of, among other things, instant copy-free moves of volumes from one DEFS to another, for example, in connection with performing load balancing. Creating clones on remote nodes for load balancing is yet another benefit. With a global PVBN space, support for global data deduplication can also be supported rather than deduplication being limited to node-level aggregates.

[0113] It is also beneficial, in terms of performance, to avoid the use of access control mechanism, such as locks, to coordinate write accesses and write allocation among nodes generally and DEFSs specifically. Such access control mechanisms may be eliminated by specifying, at a per-DEFS level, those portions of the disaggregated storage of the storage pod to which a given DEFS has exclusive write access. For example, as described further below, a DEFS may be limited to use of only the AAs associated with (assigned to or owned by) the DEFS for performing write allocation and write accesses during a CP. Advantageously, given the visibility into the entire global PVBN space, reads can be performed by any DEFS of the cluster from all the PVBNs in the storage pod.

[0114] Each DEFS of a given cluster (or data pod, as the case may be) may start at its own super block. As shown and described with reference to FIG. 6, a predefined AA (e.g., the first AA) in storage pod may be dedicated for super blocks. In one embodiment, a set of RAID stripes within the predefined super block AA (e.g., the first AA of the storage pod) may be dedicated for super blocks. In this predefined super block AA, ownership may be specified at the granularity of a single RAID stripe instead of at the AA granularity of multiple RAID stripes representing one or more GB (e.g., between approximately 1 GB and 10 GB) of storage space. The location of a super block of a given DEFS can be mathematically derived using an identifier (a DEFS ID) associated with the given DEFS. Since the RAID stripe is already reserved for a super block, it can be replicated on N disks.

[0115] Each DEFS has AAs associated with it, which may be thought of conceptually as the DEFS owning those AAs. In one embodiment, AAs may be tracked within an AA map and persisted within the DEFS filesystem. An AA map may include the DEFS ID in an AA index. While AA ownership information regarding other DEFSs in the cluster may be cached in the AA map of a given DEFS, which may be useful during the PVBN free path, for example, to facilitate freeing of PVBNs of an AA not owned by the given DEFS (which may arise in situations in which partial AAs are donated from one DEFS to another), the authoritative source information regarding the AAs owned by a given DEFS may be presumed to be in the AA map of the given DEFS.

[0116] In support of avoiding storage silos and supporting the more fluid use of disk space across all nodes of a cluster, DEFSs may be allowed to donate partially or completely free AAs to other DEFSs.

[0117] Each DEFS may have its own label information kept in the file system. The label information may be kept in the super block or another well-known location outside of the file system.

[0118] In various examples, there can be multiple DEFSs on a RAID tree. That is, there may be a many-to-one association between DEFSs and a RAID tree, in which each DEFS may have a reference on the RAID tree. The RAID tree can still have multiple RAID groups. In various

examples described herein, it is assumed the PVBN space provided by the RAID tree is continuous. [0119] It may be helpful to have a root DEFS and a data DEFS that are transparent to other subsystems. These DEFSs may be useful for storing information that might be needed before the file system is brought online. Examples of such information may include controller (node) failover (CFO) and storage failover (SFO) properties/policies. HA is one example of where it might be helpful to bring up a controller (node) failover root DEFS first before giving back the storage failover data DEFSs. HA coordination of bringing down a given DEFS on takeover/giveback may be handled by the file system (e.g., WAFL) since the RAID tree would be up until the node is shutdown.

[0120] DEFS data structures (e.g., DEFS bit maps at the PVBN level, which may also be referred to herein as “PVBN-based bitmaps,” such as active maps and reference count (refcount) maps) may be sparse. That is, they may represent the entire global PVBN space, but only include valid truth values for PVBNs of AAs that are owned by the particular DEFS with which they are associated. When validation of these bit maps is performed by or on behalf of a particular DEFS, the bits should be validated only for the AA areas owned by the particular DEFS. When using such sparse data structures, to get the complete picture of the PVBN space, the data structures in all of the nodes should be taken into consideration. While various DEFS data structures may be discussed herein as if they were separate metafiles, it is to be appreciated, given the visibility by each node into the entire global PVBN space, one or more of such DEFS data structures may be represented as cluster-wide metafiles. Such a cluster-wide metafile may be persisted in a private inode space that is not accessible to end users and the relevant portions for a particular DEFS may be located based on the DEFS ID of the particular DEFS, for example, which may be associated with the appropriate inode (e.g., an L0 block). Similarly, the entirety of such a cluster-wide metafile may be accessible based on a cluster ID, for example, which may be associated with a higher-level inode in the hierarchy (e.g., an L1 block). In any event, each node should generally have all the information it needs to work independently until and unless it runs out of storage space or meets a predetermined or configurable threshold of a storage space metric (e.g., a free space metric or a used space metric), for example, relative to the other nodes of the cluster. At that point, as described further below, as part of a space monitoring and/or a space balancing process, the node may request a portion of AAs of DEFSs owned by one or more of such other nodes be donated so as to increase the useable storage space of one or more DEFSs of the node at issue.

[0121] FIG. 6A is a block diagram illustrating a distributed storage system architecture **600** that provides disaggregated storage in accordance with an embodiment of the present disclosure. Various architectural advantages of the proposed distributed storage system architecture and mechanisms for providing and making use of disaggregated storage include, but are not limited to, the ability to perform automatic space balancing among DEFSs, perform elastic node growth and shrinkage for a cluster, perform clastic storage growth of the storage pod, perform zero-copy file and volume move (migration), perform distributed RAID rebuild, achieve HA cost reduction using volume rehosting, create remote clones, and perform global data deduplication.

[0122] In the context of the present example, the nodes (e.g., node **610a** and **610b**) of a cluster, which may represent a data pod or include multiple data pods, each include respective data dynamically extensible file systems (DEFSs) (e.g., data DEFS **620a** and data DEFS **620b**) and respective log DEFSs (e.g., log DEFS **625a** and log DEFS **625b**). In general, data DEFSs may be used for persisting data on behalf of clients (e.g., client **180**), whereas log DEFSs may be used to maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP.

[0123] It should be noted that while for simplicity only two nodes, which may be configured as part of an HA pair for fault tolerance and nondisruptive operations, are shown in the illustrative cluster depicted in FIG. 6A, there may be one or more additional nodes in a given cluster. For example, there may be multiple HA pairs within a cluster (or a data pod of the cluster, which may

represent a mechanism to limit the fault domain). As such, the description of this two-node cluster should be taken as illustrative only. Furthermore, while in some examples HA may be achieved by defining pairs of nodes within a cluster as HA partners (e.g., with one node designated as the primary node and the other designated as the secondary), in alternative examples any other node within a cluster may be allowed to step in after a failure of a given node without defining HA pairs. [0124] As discussed above, one or more volumes (e.g., volumes **630a-m** and volumes **630n-x**) or LUNs (not shown) may be created by or on behalf of customers for hosting/storing their enterprise application data within respective DEFSs (e.g., data DEFSs **620a** and **620b**).

[0125] While additional data structures may be employed, in this example, each DEFS is shown being associated with respective AA maps (indexed by AA ID) and active maps (indexed by PVBN). For example, log DEFS **625a** may utilize AA map **627a** to track those of the AAs within a global PVBN space **640** of storage pod **645** (which may be analogous to storage pod **145**) that are owned by log DEFS **625a** and may utilize active map **626a** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; log DEFS **625b** may utilize AA map **627b** to track those of the AAs within the global PVBN space **640** that are owned by log DEFS **625b** and may utilize active map **626b** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; data DEFS **620a** may utilize AA map **622a** to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620a** and may utilize active map **621a** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; and data DEFS **620b** may utilize AA map **622b** to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620b** and may utilize active map **621b** to track at a PVBN level of granularity which of the PVBNs of its AAs are in use.

[0126] In this example, each DEFS of a given node has visibility and accessibility into the entire global PVBN address space **640** and any AA (except for a predefined super block AA **642**) within the global PVBN address space **640** may be assigned to any DEFS within the cluster. By extension, each node has visibility and accessibility into the entire global PVBN address space **640** via its DEFSs. As noted above, the respective AA maps of the DEFSs define which PVBNs to which the DEFSs have exclusive write access. AAs within the global PVBN space **640** shaded in light gray, such as AA **641a**, can only be written to by node **610a** as a result of their ownership by or assignment to data DEFS **620a**. Similarly AAs within the global PVBN space **640** shaded in dark gray, such as AA **641b**, can only be written to by node **610b** as a result of their ownership by or assignment to data DEFS **620b**.

[0127] Returning to super block **642**, it is part of a super block AA (or super AA). In the context of FIG. 6A, the super AA is the first AA of the storage pod **645**. The super AA is not assigned to any DEFS (as indicated by its lack of shading). The super AA may have an array of DEFS areas which are dedicated to each DEFS and can be indexed by a DEFS ID. The DEFS ID may start at index 1 and in the context of the present example includes four super block and four DEFS label blocks. The DEFS label can act as a RAID label for the DEFS and can be written out of a CP and can store information that needs to be kept outside of the file system. In a pairwise HA configuration, two super blocks and two DEFS label blocks may be used by the hosting node and the other two may be used by the partner node on takeover. Each of these special blocks may have their own separate stripes.

[0128] In the context of the present example, it is assumed after establishment of the disaggregated storage within the storage pod **645** and after the original assignment of ownership of AAs to data DEFS **620a** and data DEFS **620b**, some AAs have been transferred from data DEFS **620a** to data DEFS **620b** and/or some AAs have been transferred from data DEFS **620b** to data DEFS **620a**. As such, the different shades of grayscale of entries within the AA maps are intended to represent potential caching that may be performed regarding ownership of AAs owned by other DEFSs in the cluster. For example, assuming ownership of a partial AA has been transferred from data DEFS **620a** to data DEFS **620b** as part of an ownership change performed in support of space balancing,

when data DEFS **620a** would like to free a given PVBN (e.g., when the given PVBN is no longer referenced by data DEFS **620a** as a result of data deletion or otherwise), data DEFS **620a** should send a request to free the PVBN to the new owner (in this case, data DEFS **620b**). This is due to the fact that in various embodiments, only the current owner of a particular AA is allowed to perform any modify operations on the particular AA. Further explanation regarding space balancing and AA ownership change is provided below with reference to FIG. **9** and FIG. **10**, respectively.

[0129] Those skilled in the art will appreciate disaggregation of the storage space as discussed herein can be leveraged for cost-effective scaling of infrastructure. For example, the disaggregated storage allows more applications to share the same underlying storage infrastructure. Given that each DEFS represents an independent file system, the use of multiple of such DEFSs combine to create a cluster-wide distributed file system since all of the DEFSs within a cluster share a global PVBN space (e.g., global PVBN space **640**). This provides the unique ability to independently scale each independent DEFS as well as enables fault isolation and repair in a manner different from existing distributed file systems.

[0130] Additional aspects of FIG. **6A** will now be described in connection with a discussion of FIG. **6B**, which represents a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod (e.g., storage pod **645**). The processing described with reference to FIG. **6B**, may be performed by a combination of a file system (e.g., file system **360**) and a RAID system (e.g., RAID system **380**), for example, during or after an initial boot up.

[0131] At block **661**, the storage pod is created based on a set of disks made available for use by the cluster. For example, a job may be executed by a management plane of the cluster to create the storage pod and assign the disks to the cluster. Depending on the particular implementation and the deployment environment (e.g., on-prem versus cloud), the disks may be associated with one or more disk arrays or one or more storage shelves or persistent storage in the form of cloud volumes provided by a cloud provider from a pool of storage devices within a cloud environment. For simplicity, cloud volumes may also be referred to herein as “disks.” The disks may be HDDs or SSDs.

[0132] At block **662**, the storage space of the set of disks may be divided or partitioned into uniform-sized AAs. The set of disks may be grouped to form multiple RAID groups (e.g., RAID group **650a** and **650b**) depending on the RAID level (e.g., RAID 4, RAID 5, or other). Multiple RAID stripes may then be grouped to form individual AAs. As noted above, an AA (e.g., AA **641a** or AA **641b**) may be a large chunk representing one or more GB of storage space and preferably accommodates multiple SSD erase blocks worth of data. In one embodiment, the size of the AAs is tuned for the particular file system. The size of the AAs may also take into consideration a desire to reduce the need for performing space balancing so as to minimize the need for internode (e.g., East-West) communications/traffic. In some examples, the size of the AAs may be between about 1 GB to 10 GB. As can be seen in FIG. **6A**, dividing the storage pod **645** into AAs allows available storage space associated with any given disk or any RAID group to be use across many/all nodes in the cluster without creating silos of space in each node. For example, at the granularity of an individual AA, available storage space within the storage pod **645** may be assigned to any given node in the cluster (e.g., by way of the given node's DEFS(s)). For example, in the context of FIG. **6A**, AA **641a** and the other AAs shaded in light gray are currently assigned to (or owned by) data DEFS **620a** (which has a corresponding light gray shading). Similarly, AA **641b** and the other AAs shaded in dark gray are currently assigned to (or owned by) data DEFS **620b** (which has a corresponding light gray shading).

[0133] At block **663**, ownership of the AAs is assigned to the DEFSs of the nodes of the cluster. According to one embodiment, an effort may be made to assign group of consecutive AAs to each DEFS. Initially, the distribution of storage space represented by the AAs assigned to each type of DEFS (e.g., data versus log) may be equal or roughly equal. Over time, based on differences in storage consumption by associated workloads, for example, due to differing write patterns,

ownership of AAs may be transferred among the DEFSs accordingly.

[0134] As a result, of creating and distributing the disaggregated storage across a cluster in this manner, all disks and all RAID groups can theoretically to be accessed concurrently by all nodes and the issue discussed with reference to FIG. 5 in which the entirety of any given disk and the entirety of any given RAID group is owned by a single node is avoided.

Example Automatic Storage Space Balancing Among Dynamically Extensible File Systems

[0135] FIG. 7A is a block diagram illustrating two dynamically extensible file systems (DEFSs)

720a and **720b** prior to performance of automatic space balancing in accordance with an embodiment of the present disclosure. In one embodiment, each DEFS of a cluster may track a storage space metric (e.g., free space or used space) and may periodically broadcast a space report to all other nodes of the cluster. A non-limiting example of such space monitoring is described further below with reference to FIG. 8. As described further below with reference to FIG. 9, cluster-wide space reporting may be used to perform automatic space balancing so as to allow DEFSs running low on free space relative to other DEFSs in the cluster to receive free or partial AAs from one or more other DEFSs in the cluster.

[0136] In the context of the present example, DEFS **720a** (which may be analogous to data DEFS **620a**) is shown with a greater amount of used storage space and a lesser amount of free storage space than DEFS **720b** (which may be analogous to data DEFS **620b**). At this point in time, AA map **722a** of DEFS **720a** is shown with ten AA entries shaded in light gray, indicating it owns ten AAs, six of which may be used; and AA map **722b** of DEFS **720b** is shown with ten AA entries shaded in dark gray, indicating it owns ten AAs, only two of which may be used.

[0137] Based on cluster-wide space reporting, DEFS **720a** may request a specified amount of storage space from all other nodes in the cluster. Assuming for sake of simplicity, DEFS **720a** and DEFS **720b** are part of a two-node cluster and are the only DEFSs in the cluster, the request for space made from the node owning DEFS **720a** to the node owning DEFS **720b** may attempt to create a more equal amount of free storage space available to each DEFS as shown in FIG. 7B.

[0138] While in the context of the present example, one specific type of space reporting is described via push communication (i.e., periodic broadcasting of space reports), it is to be appreciated other types of push communication approaches may be used. For example, DEFSs may perform space reporting responsive to a trigger event (e.g., a change in the storage space metric by a threshold since the last space report was sent). As yet another alternative, the storage space reporting may be via a pull communication approach, for example, in which nodes or DEFSs, periodically or responsive to a trigger event, poll the others for their respective storage space metrics.

[0139] FIG. 7B is a block diagram illustrating the two dynamically extensible file systems (DEFSs) **720a** and **720b** of FIG. 7A after completion of automatic space balancing in accordance with an embodiment of the present disclosure. In this example, the size of the triangle representing DEFS **720b** has shrunk to represent its total available storage space has been reduced as a result of donating one or more AAs to DEFS **720a**. The size of triangle representing DEFS **720a** has grown to represent its total available storage space has been increased. So the total used space of the DEFSs remains the same, but the amount of free space has been balanced. Assuming DEFS **720a** requested two AAs worth of storage space from DEFS **720b** and DEFS **720b** accepted this request, it can now be seen (based on the change in AA maps) that upon completion of the automatic space balancing, DEFS **720a** now owns twelve AAs and DEFS **720b** now owns eight AAs and the free space available to both DEFSs is now more balanced.

Space Monitoring

[0140] FIG. 8 is a flow diagram illustrating operations for performing space monitoring in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. 8 may be performed by a storage system (e.g., node **110a**, **110b**, **610a**, or **610b**) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **610a**, **610b**, and possibly

one or more other nodes).

[0141] At decision block **805**, a determination is made regarding the trigger event that activated the space monitoring process. If the trigger event represents expiration of a timer representing a space reporting interval (which may be measured in minutes), processing continues with block **810**. If the trigger event represents an indication that a write is being performed to a block within the PVBN space of the AAs owned by a DEFS of the node, the processing proceeds with block **830**. If the trigger event represents expiration of a timer representing a space balancing check interval (which may be measured in minutes), processing continues with block **850**. If the trigger event represents receipt of a space report from another node in the cluster, processing branches to block **890**.

[0142] At block **810**, the current node's free space metric is updated. For a given node, its free space metric may be calculated by subtracting the total used PVBN space (e.g., the sum of PVBN counts for all AAs of all DEFSs of the node multiplied by the block size) for the node from the total useable space for all of its DEFSs. Assuming a given DEFS has N AAs, each of having Y PVBNs, the total usable space of the given DEFS may be calculated as $N \times Y \times \text{the block size}$.

[0143] In one embodiment, ownership information regarding DEFSs and nodes may be maintained within a cluster configuration database (e.g., a replicated database (RDB)) accessible by all nodes, for example, stored on the boot media (e.g., boot disks) and maintained by a management plane of the cluster. The ownership information may include information regarding which DEFSs (e.g., as identified by their respective DEFS IDs) are owned by a given node (e.g., as identified by its node ID), for a given DEFS which node hosts it (or owns it). Additionally, nodes may maintain respective in-memory hash tables (e.g., a DEFS ownership hash table and an AA ownership hash table) to determine whether a given DEFS is owned by the node and whether a given AA is owned by a DEFS of the node. The cluster configuration database may also include other information, including volume names and/or volume identifiers (IDs).

[0144] At block **820**, the node may contribute to cluster-wide space reporting by incorporating the node-level free space metric calculated in block **810** into a space report message and broadcasting the space report message to all other nodes of the cluster. After completion of the space reporting, processing loops back to decision block **805** to await the next trigger event.

[0145] At block **830**, one or more corresponding PVBN entries within the DEFS's active map (e.g., one of active maps **621a** or **622a**) are updated, for example, during the write path to mark the PVBN(s) affected by the write operation as in use.

[0146] At block **840**, a used (or in-use) PVBN count, indicative of how many PVBNs of the total number of PVBNs available to the DEFS within its total AA ownership PVBN space is updated. After completion of block **840**, processing loops back to decision block **805** to await the next trigger event.

[0147] At block **850**, the current node's free space metric is updated as described above with respect to block **810**.

[0148] At block **860**, the current node's free space metric may be compared to that of the other node(s) of the cluster (as reported by them, for example, on a periodic basis in accordance with the space reporting interval). According to one embodiment, an average amount of free space per node may be calculated. For example, the amount of free space cluster-wide (based on a sum of all the reported free space by all nodes in the cluster) may be determined and divided by the number of nodes in the cluster. This average amount of free space per node may then be compared to the current node's free space metric.

[0149] At decision block **870**, it is determined whether the result of the comparison performed in block **860** should result in triggering of space balancing. If so, processing continues with block **880**; otherwise, processing loops back to decision block **805** to await the next trigger event. In one embodiment, space balancing is triggered responsive to the average amount of free space per node exceeding the current node's free space metric by a predetermined or configurable threshold.

[0150] While in the context of the present example, a free storage space metrics are used as a space

balancing trigger, it is to be appreciated a used storage space metric may alternatively be used. Similarly, while the cluster-wide space reporting is described as being performed at the node-level in the context of the present example, it is to be appreciated such cluster-wide space reporting may alternatively be performed at the DEFS level.

Space Balancing

[0151] FIG. 9 is a flow diagram illustrating operations for performing space balancing in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. 9 may be performed by a storage system (e.g., node **110a**, **110b**, **610a**, or **610b**) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **610a**, **610b**, and possibly one or more other nodes).

[0152] At decision block **905**, a determination is made regarding the trigger event that activated the space balancing process. If the trigger event represents receipt of a request for space, processing continues with decision block **910**. If the trigger event represents expiration of a timer for a space request previously sent by the current DEFS, then processing continues with decision block **930**. If the trigger event represents receipt of a rejection of a space request previously sent by the current DEFS, the processing continues with block **950**. If the trigger event represents receipt of an acceptance of a space request previously sent by the current DEFS, the processing continues with block **960**.

[0153] At decision block **910**, it is determined whether sufficient space is available for the current DEFS to donate one or more of its AAs to the requester. If so, processing continues with block **920**. According to one embodiment, as part of the determination regarding whether it has sufficient storage space available, the current DEFS may take into consideration how much of its storage space should be held in reserve for CP usage, for example, based on the write workload on the node that owns the current DEFS. Another consideration may be whether the DEFS has within a predetermined or configurable timeframe already donated one or more of its AAs to the current requester or to another requester.

[0154] At block **920**, the current DEFS attempts to perform an AA ownership change of one or more of its AA from the current DEFS to the requester. In one embodiment, the current DEFS selects a number of its best owned AAs for the ownership change. In one embodiment, the “best” AAs may be selected by prioritizing the donation of free AAs over partial AAs as donation of partial AAs comes with the additional overhead of copying the associated bit maps or portions thereof (e.g., active map, AA info file, etc.). However, if an insufficient number of free AAs are available to meet the storage space needs of the requester as expressed by the received request for storage, the current DEFS may fulfill the request by selecting one or more partial AAs to be donated to the requester. Additionally, to support efficient performance of CPs, the current DEFS may also select for donation, those AAs that will not create a discontinuity or minimize discontinuities within a set of consecutive AAs. A non-limiting example of AA ownership change processing is described below with reference to FIG. 10. After completion of the AA ownership change processing, processing loops back to decision block **905** to await the next trigger event.

[0155] At decision block **930**, it is determined whether the space request at issue previously made by the current DEFS has been accepted or rejected. If the space request was accepted or rejected by the source DEFS (the donor DEFS), which may be determined based on logging of receipt of such responses, then processing loops back to decision block **905** to await the next trigger event; otherwise, if no response has been received, then processing branches to block **940**.

[0156] At block **940**, the request for space may be retried. Those skilled in the art will be familiar with various retry mechanisms, including handling exceptions that might occur in re-processing the messages. For example, delayed queues and Dead-letter queues are two such data structures that can be used to implement a retry mechanism effectively. After completion of the retry, processing loops back to decision block **905** to await the next trigger event.

[0157] At block **950**, the rejection of the space request is logged so as to stop retries. After

completion of block **950**, processing loops back to decision block **905** to await the next trigger event.

[0158] At block **960**, the acceptance of the space request is logged to stop retries. In one embodiment, the acceptance may be in the form of an AA ownership change request message from the source (or donor) DEFS. An AA ownership change request message may include the AA IDs of the AAs. Based on the AA IDs contained in the AA ownership change request message, the current DEFS may update its AA map to reflect the fact that the current DEFS now owns the AAs donated by the source (or donor) DEFS.

[0159] At block **970**, the current DEFS may update its free space metric to account for the new storage space available to it as a result of the newly owned AAs.

[0160] At block **980**, the node may perform a consistency point to persist the updated AA data structures to disk.

[0161] At block **990**, the current DEFS may send a reply back to the source (or donor) DEFS with retry to indicate acceptance of the AA ownership change request. After completion of block **990**, processing loops back to decision block **905** to await the next trigger event.

Allocation Area Movement

[0162] FIG. **10** is a flow diagram illustrating operations for attempting to perform allocation area ownership change in support of space balancing in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. **10** may be performed by a storage system (e.g., node **110a**, **110b**, **610a**, or **610b**) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **610a**, **610b**, and possibly one or more other nodes). It is assumed prior to attempting to perform an AA ownership change from the source DEFS (which may also be referred to herein as the donor or transferor DEFS), one or more AAs (e.g., free AAs and/or partial AAs) have previously been selected that are to be donated or transferred to the destination (which may also be referred to herein as the donee or recipient DEFS).

[0163] At block **1005**, the AA ownership of the selected AAs, for example, maintained in an AA ownership map (e.g., AA map **622b** or **722b**) maintained by the source DEFS (e.g., data DEFS **620b** or **720b**) is updated. For example, the AAs selected for transfer from the source DEFS to the destination DEFS (e.g., data DEFS **620a** or **720a**) may be removed from the AA map of the source DEFS and their respective AA IDs may be added to an ownership change or transfer message.

[0164] At block **1010**, the AA ownership of the selected AAs and an associated PVBN bit map (e.g., active map **621b**) are transferred from the source DEFS to the destination DEFS, for example, by adding the ownership change or transfer message to an outbound queue (e.g., an outbound persistent message queue associated with the source DEFS or the node hosting the source DEFS). During block **1005** or **1010**, the associated PVBN bit map may also be frozen to temporarily preclude use of the PVBN bit map since the source DEFS no longer owns the selected AAs that are in the process of being transferred to the destination DEFS.

[0165] In the context of the present example, the ownership transfer is accomplished by way of a local update of AA ownership information and a persistent message added to a persistent message queue during the same CP (e.g., CP X) of the source DEFS. Depending on the particular implementation, copies of messages in outbound queues may be copied to the inbound queues of the intended recipients on a periodic basis or in response to a trigger event.

[0166] At decision block **1015**, the destination DEFS determines whether there is a new message (e.g., the ownership change or transfer message generated by the source DEFS in blocks **1005** and/or **1010**) in its inbound queue. If so, processing continues with decision block **1020**; otherwise, processing loops back to decision block **1015**. In one embodiment, a polling thread may be implemented for each inbound queue and may periodically check the inbound queue for the presence of a message on the inbound queue.

[0167] At decision block **1020**, a message handling result is determined. If the ownership transfer of the AAs from the source DEFS is rejected by the destination DEFS, for example, as a result of a

sufficient number of AAs already having been received from one or more other DEFSs of the cluster, then processing continues with block **1025**; otherwise, if the ownership transfer of the AAs from the source DEFS is accepted by the destination DEFS, the processing branches to block **1035**. [0168] At block **1025**, the ownership of the AAs is returned to the source DEFS, for example, by adding the ownership change or transfer message just received from the source DEFS to the outbound queue of the destination DEFS with the destination updated to reference the DEFS ID of the source DEFS.

[0169] At block **1030**, the inbound queue of the destination DEFS is updated. For example, depending on the particular implementation of the persistent message queues, a head pointer for the inbound queue may be updated or a sequence number of the message may be updated (e.g., incremented).

[0170] At block **1035**, the PVBN bit maps (e.g., active maps **621a** and **621b**) are updated. For example, those portions of the PVBN bit map of the source DEFS associated with the transferred AAs identified within the ownership transfer message may be moved or copied to the PVBN bit map of the destination DEFS. As should be appreciated the PVBN bit map associated with the AAs involved in the ownership transfer should not be used until ownership of the AAs identified by the ownership change or transfer message has been accepted and updated by the destination DEFS or rejected and re-assigned on the source DEFS. At that point, the PVBN bit map may be unfrozen to allow usage of the PVBN bit map to resume.

[0171] At block **1040**, the AA ownership of the AAs being transferred to the destination DEFS is updated, for example, by updating an AA ownership map (e.g., AA map **622a** or **722a**) maintained by the destination DEFS.

[0172] In the context of the present example, the ownership transfer processing on the destination DEFS is performed by way of a local update of AA ownership information and updating of its inbound queue during the same CP (e.g., CP N) of the destination DEFS.

[0173] At decision block **1050**, it is determined whether the ownership change or transfer message originated by the source DEFS has been processed. If so, processing continues with block **1055**; otherwise, processing loops back to decision block **1050**. In one embodiment, a polling thread may be implemented for each outbound queue. For example, the polling thread associated with the outbound queue of the source DEFS may periodically check the intended recipient's inbound queue (in this case, the inbound queue of the destination DEFS) for an indication that processing of a given message (in this case, the ownership change or transfer message) has been completed by the intended recipient. Depending on the particular persistent message queue implementation, the completion of the processing of the ownership change or transfer message may be detected, for example, based on observing an update to the head pointer of the inbound queue of the destination DEFS or based on observing a change to the sequence number of the message in the inbound queue of the destination DEFS.

[0174] At block **1055**, the ownership change or transfer message is removed from the outbound persistent message queue of the source DEFS and another CP is taken (e.g., CP Y).

[0175] In the context of the present example, it is to be noted that AA movement (transfer of ownership) is made possible by moving one or more bit maps relating to PVBN ownership (e.g., active maps **621a** or **621b**) from one DEFS to another. Modification to a given AA map (e.g., AA map **622a** or **622b**) changes the ownership of the AA at issue; however, which PVBNs are still in use is identified and tracked by the PVBN bit maps (e.g., active map **621a** or **621b** and refcount maps) so moving part of such bit maps from one DEFS to another facilitates movement of partial AAs. In one embodiment, the movement of the PVBN bit maps along with the AA movement may be done in lock step with a CP so as to avoid loss of any data in the transfer.

[0176] Once a given AA is moved from one DEFS to another, freeing of PVBNs is collected and moved to the new owner DEFS to decrement the refcount. Since, in one embodiment, all the modify operations of AA data structure may only be performed by the DEFS that owns the AA at

issue, there is no need for locking.

[0177] While in the context of the present example, a persistent message queue is assumed to be used to perform inter-node communications, it is to be understood on-wire transfers (e.g., network communications), for example, via RPCs may alternatively be used.

Removal of a Node

[0178] FIG. **11** is a flow diagram illustrating operations for performing space balancing responsive to removal of a node from a cluster in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. **11** may be performed by the coordinated actions of the DEFS(s) (e.g., log DEFS **625a** and data DEFS **620a** or log DEFS **625b** and data DEFS **620b**) associated with the node (e.g., node **610a** or **610b**) to be removed, one or more other DEFSs within the cluster, the node to be removed, and the node(s) in the cluster hosting the one or more other DEFSs. In the context of the present example, it is assumed the cluster may operate in accordance with one or more modes of operation in connection with removal of a node. For example, the cluster may operate in a single, preset mode of operation involving parking one or more DEFSs owned by the node to be removed within another node of the cluster, a single, preset mode of operation involving deletion of the one or more DEFSs, or both modes of operation may be available and administrative input or configuration parameters associated with the cluster may select the mode of operation.

[0179] At decision block **1110**, a determination is made regarding the mode of operation of the cluster. If the mode of operation relates to parking of the one or more DEFSs owned by the node to be removed within another node of the cluster, then processing continues with block **1120**; otherwise, if the mode of operation relates to deletion of the one or more DEFSs, then processing branches to block **1140**.

[0180] At block **1120**, volume(s) (e.g., volumes **630a-m** or volumes **630n-x**) associated with the DEFS(s) of the node being removed are distributed to one or more other DEFSs in the cluster that are owned by one or more other nodes. According to one embodiment, the movement of a volume from one DEFS to another may be performed without copying the data stored within the volume. For example, a container file inode may be created on the destination DEFS and the top-most container file PVBN of the container file inode on the source DEFS may simply be moved to the new inode. Since all the PVBNs are accessible from all DEFSs in a cluster the just moving the top-most PVBN is good enough. This efficient volume movement may be referred to herein as “zero-copy volume move” or the like.

[0181] At block **1130**, most of the AAs owned by the DEFS(s) of the node that is being removed are distributed to the one or more other node(s) of the cluster. This distribution of AAs may be performed via the AA ownership change mechanism described earlier, for example, with reference to FIGS. **9** and **10**. In one embodiment, some small number of AAs may be maintained by the DEFS(s) to avoid certain complexities associated with attempting to transfer ownership of all AAs, including atomically updating the associated storage space information with the file system via a CP, which itself needs AAs to persist data to the storage pod (e.g., storage pod **645**).

[0182] At block **1135**, the DEFS(s) of the node that is being removed are parked within another node of the cluster by changing ownership of the DEFS(s) to the other node. In one embodiment, as the parked DEFS(s) still have a few AAs, they will remain online; however, as the data associated with the parked DEFS(s) is not expected to change, there should be no need for the parked DEFS(s) to go through CPs. The ownership of a given parked DEFS may be subsequently reassigned to yet another node, for example, responsive to a request by the other node to create a new DEFS. As such, parking a DEFS avoid the overhead associated with deleting the DEFS and may facilitate more efficient workflow associated with new DEFS creation.

[0183] At block **1140**, volume(s) (e.g., volumes **630a-m** or volumes **630n-x**) associated with the DEFS(s) of the node being removed are distributed to one or more other DEFSs in the cluster that are owned by one or more other nodes as described above with reference to block **1120**.

[0184] At block **1150**, the AAs owned by the DEFS(s) of the node that is being removed are distributed to the one or more other node(s) of the cluster. This distribution of AAs may be performed as described above with reference to block **1130**; however, since the DEFS(s) are to be deleted, the ownership of all AAs of the DEFS(s) at issue should be transferred, including atomically updating the associated storage space information with the file system via a CP.

[0185] At block **1160**, the DEFS(s) of the node that is being removed can now be deleted.

Addition of a New Node

[0186] FIG. **12** is a flow diagram illustrating operations for performing space balancing responsive to addition of a new node to a cluster in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. **12** may be performed by the coordinated actions of the management plane of the cluster, an existing node (e.g., one of nodes **610a** or **610b**) of the cluster, and one or more DEFSs (e.g., log DEFS **625a** and data DEFS **620a** or log DEFS **625b** and data DEFS **620b**) associated with the existing nodes (e.g., node **610a** or **610b**), the newly added node, and the one or more new DEFSs of the new node. Prior to starting at block **1210**, it is assumed a new storage system is available to be added to the cluster, for example, as a result of creating a new instance of a node within a cloud environment based on available resources (e.g., processing resources, such as one or more CPUs)

[0187] At block **1210**, the process of creating one or more new DEFSs for the new node is started. In one embodiment, during a cluster join process, for example, performed by the management plane of the cluster to add the new node to the cluster, new DEFS IDs may be identified and assigned to the new DEFSs to be created for the new node. The new DEFS(s) may first be created on an existing node (e.g., node **610a** or **610b**) of the cluster with a few AAs donated from one or more existing DEFSs (e.g., log DEFS **625a** and data DEFS **620a** or log DEFS **625b** and data DEFS **620b**) owned by the existing node. As part of the creation of the new DEFS(s), new data structures (e.g., AA maps and other bit maps) may also be created for the new DEFS(s). In one embodiment, the default set of DEFSs for a new node includes at least one log DEFS and at least one data DEFS. Since the donated AAs are from one or more DEFSs on the same node (the existing node) as the node on which the new set of one or more DEFSs is being created, in one embodiment, the messaging-based or RPC-based approach described above in connection with performing AA ownership changes may be avoided and instead a file system message may be used to perform the desired AA ownership changes.

[0188] At block **1220**, the newly created set of one or more DEFS(s) created on behalf of the new node by the existing node of the cluster are taken offline and the ownership of the new DEFS(s) is transferred to the new node.

[0189] After completion of block **1220**, the new node has a set of one or more newly created DEFSs that each have a few AAs. At this point, additional AAs may be added to each of the one or more newly created DEFSs.

[0190] At block **1230**, as a result of periodic space monitoring performed by all nodes and/or DEFSs of the cluster, for example, as described above with reference to FIG. **8**, the new node will identify a need for more storage space on the part of its new DEFS(s) and issue request(s) for space to one or more other nodes of the cluster.

[0191] At block **1240**, as a result of space balancing, for example, as described above with reference to FIG. **9**, performed responsive to the request(s) for space issued in block **1230**, the new DEFS(s) will be assigned ownership of additional AAs previously owned by one or more other DEFS(s) within the cluster.

[0192] Elastic node growth and shrinkage can facilitate implementation of a pay-as-you-go storage service delivery model. For example, when additional infrastructure resources are desired or existing infrastructure resources are no longer needed, they can easily be added or removed and the customer may be charged on the basis of their usage.

[0193] It is to be noted that, apart from adding a node to scale out (or horizontally scale) the cluster

for purposes of handling storage operations, the incremental addition of nodes to a cluster may also be used for other data management operations or compute-intensive data services. For example, a dedicated server (or compute) node with an x86 or x64 instruction set architecture and one or more CPUs and/or one or more graphics processing units (GPUs), having a light-weight data adaptor (rather than the full storage operating system) to facilitate access to the storage pod via a backend storage network, may be added to a cluster to perform data services (sometimes described as Data-as-a-Service). The category of data services is quite broad. Data services can help with the aggregation of data from various parts of a storage architecture, or in the creation of a central data center repository. Data services may deal with data in transit, or with storage. Data services could also perform various types of analytics, including, for example, artificial-intelligence (AI)-powered data analytics, on big data sets. Data services may refer to various categories of software that facilitate access to data, and/or analysis of data. Non-limiting examples of data services include small, independent, and loosely coupled functions that enhance, organize, share, or calculate information collected and saved in data storage volumes. Data services may amplify traditional data by improving its resiliency, availability, and validity, as well as adding characteristics to data that it doesn't already have natively-like metadata. Non-limiting examples of data services include, but are not limited to performance of file system analytics and cataloging of user data assets. Additionally or alternatively, the incremental addition of nodes to a cluster may be used to support various compliance and/or security services (e.g., anti-ransomware and/or ransomware recovery) without impacting the core storage performance provided by the other nodes of the cluster running a full storage operating system.

Example Zero-Copy Volume Move

[0194] FIG. **13A** is a block diagram conceptually illustrating an initial state of a source node **1310a** and a destination node **1310b** prior to performance of a zero-copy volume move in accordance with an embodiment of the present disclosure. Source node **1310a** may be analogous to node **610a** and destination node **1310b** may be analogous to node **610b**. In the context of the present example, source node **1310a** and destination node **1310b** are shown each including only a single DEFS (e.g., DEFS **1320a** and **1320b**, respectively). In this initial state, DEFS **1320b** (which represents the destination of a volume move) is shown having PVBN-based bitmaps **1340b** but no volumes.

[0195] DEFS **1320a** (which represents the source of the volume move) includes two volumes (e.g., volumes **1330a** and **1330b**) and corresponding PVBN-based bitmaps **1340a**. The top-most portion of the file system tree (e.g., corresponding to the inode file data block of FIG. **4** containing container file inode **431**) associated with volume **1330b** (the volume that will be moved from the source node **1310a** to the destination node **1310b** in this example) is shown including the volume directory and inodes associated with the volume **1330b**. In one embodiment, based on the hierarchical file system layout described above with reference to FIG. **4**, the volume directory for a given volume may include a container file inode and one or more other inodes associated with metadata files that may store various properties of the given volume. The container file inode may point to the top-most directories of the given volume. The inodes of each directory within the given volume in turn point may point to respective sub-directories, The inodes of each sub-directory within a given directory may point to respective files. As such, it will be appreciated, in the context of a distributed storage system that makes use of a storage pod (e.g., storage pod **145**), for example, in which all nodes of the distributed storage system have visibility into a global PVBN space via their respective DEFSs, the simple act of moving the block content (e.g., metadata containing the top-most PVBNs of a given volume) of the disk inode of the container file inode that hosts the given volume may be used to make the given volume accessible on the node to which such block content is moved.

[0196] In the context of this example, the use of light-gray shading is intended to indicate the use of PVBNs from an AA owned by DEFS **1320a** and the use of dark-gray shading is intended to indicate the use of PVBNs from an AA owned by DEFS **1330a**.

[0197] FIG. 13B is a block diagram conceptually illustrating a state of the source node and the destination node of FIG. 13A after completion of the zero-copy volume move in accordance with an embodiment of the present disclosure. As described further below, with reference to FIG. 15, before transferring a given volume (e.g., volume 1330b) to the destination node 1310b, a new volume directory will be created within the DEFS (e.g., DEFS 1320b) of the destination node (e.g., destination node 1310b). As the new volume directory uses PVBNs of AAs of DEFS 1320b, the top-most portion of volume 1330b is depicted with dark gray shading. In one embodiment, the zero-copy volume move involves copying of metadata associated with the volume being moved but no copying of the volume data. For example, the block content (e.g., metadata containing the top-most PVBNs of the volume) of the disk inode of the container file inode that hosts the volume may be transferred (e.g., via a persistent message queue mechanism or on-wire communication) from the source node 1310a to the destination node 1310b. To the extent property information associated with the volume is maintained outside of the container file representing the volume, for example, in metafiles within the volume directory, such additional metadata may be copied or moved in a similar manner to the zero-copy volume move. This may be a design decision or conditional logic made based on various factors including the size of the individual metafiles.

[0198] One point to be noted in this example is the volume metafile indirects and L0 blocks (shaded in light gray) of volume 1330b remain within PVBNs owned by or assigned to DEFS 1320a, whereas new data added to volume 1330b after completion of the volume move will make use of PVBNs owned by or assigned to DEFS 1320b (shaded in dark gray). As a result, the free path workflow (e.g., on overwrite of a block or freeing of a block in the light-gray region of volume 1330b) will involve the participation of source node 1310a as the PVBN-based bitmaps 1340a (e.g., indicating the usage status of a given PVBN within AAs owned by DEFS 1320a) should be updated within DEFS 1320a. According to one embodiment, a remote PVBN free module may be implemented to send a file containing a list of PVBNs to be freed to the DEFS that owns the AA or AAs within which the PVBNs reside.

[0199] In the context of various examples, as DEFSs are able to see the entire PVBN space of a storage pod (e.g., storage pod 145), which means they are able to read any PVBN directly from any disk associated with the storage pod, this ability can be leveraged to perform zero-copy volume movement (without the need to copy the underlying volume data). In one embodiment, since each volume (e.g., flexible volume) is represented in the form of a set of files (e.g., a container file hosting the volume data and one or more metafiles storing information relating to properties of the volume) in the volume directory of a DEFS, an example of a simplified zero-copy volume move workflow would involve creating all the required inodes in the volume directory of the destination DEFS, pausing the traffic on the existing volume, and taking a final CP. At this point, the PVBN(s) at the top of the inode tree for all the directories, subdirectories, and files in the source volume directory can simply be moved to the destination volume directory. Finally, the volume can be opened for business on the destination node. Subsequent writes to the volume will use AAs owned by the destination DEFS. While load balancing is one example of a use case in which zero-copy volume move may be beneficial, those skilled in the art will appreciate other potential use cases.

[0200] FIG. 14 is a high-level flow diagram illustrating operations or performing zero-copy volume move in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. 14 may be performed by a distributed storage system (e.g., cluster 100 or a cluster including nodes 110a and 110b, 610a and 610b, or nodes 1310a and 1310b, and/or possibly one or more other nodes).

[0201] At block 1410, a storage pool is provided for use by the distributed storage system in which the storage space associated with a group of disks is visible to multiple nodes of the distributed storage system via a global PVBN space. In one embodiment, the storage pool may be represented in the form of a storage pod (e.g., storage pod 145), for example, created as described above with

reference to FIG. 6B.

[0202] At block **1420**, a source volume (e.g., volume **1330b**) of a source node (e.g., source node **1310a**) is moved to a destination node (e.g., destination node **1310b**) without performing transfer of data associated with the source volume. According to one embodiment, this zero-copy volume move performs a transfer of metadata of the source volume to the destination node by moving inode block content of a first container file representing the source volume to a new inode of a second container file representing a destination volume on the destination node. In one example, a volume directory of the source volume includes a container file inode (e.g., container file inode **431**) that hosts the data of the source volume and one or more other inodes associated with metadata files that may store various properties of the source volume. The container file inode may have stored therein pointers to the top-most directories of the given volume. As a result, the source volume may be moved from the source node to the destination node by simply moving the block content of the disk inode of the container file inode.

[0203] FIG. **15** is a more detailed flow diagram illustrating a distribution of operations between a source node and a destination node for performing zero-copy volume move in accordance with an embodiment of the present disclosure. As above, the processing described with reference to FIG. **15** may be performed by a distributed storage system (e.g., cluster **100** or a cluster including nodes **110a** and **110b**, **610a** and **610b**, or nodes **1310a** and **1310b**, and/or possibly one or more other nodes). In one embodiment, at a high-level, the proposed zero-copy volume move may involve operations performed a source node (e.g., source node **1310a**), representing the source of the volume being moved, a destination node (e.g., destination node **1310b**), representing the destination of the volume, and a cluster-level job (e.g., a cluster-level volume move job) that may coordinate the activities of the source and destination nodes, perform various configuration checks, and update appropriate information in the cluster configuration database to reflect, among other things, the new ownership of the moved volume upon completion of the cutover from the source volume to a new destination volume on the destination node.

[0204] At block **1505**, before the transfer of the volume starts, the destination node creates a new destination volume. In one embodiment, the destination volume is created as a zero-sized (empty) container file inode within a new volume directory of a destination DEFS (e.g., DEFS **1320b**) of the destination node and creation of all desired volume metafile inodes within the new volume directory. At this time, the destination volume may also at be marked as restricted.

[0205] At block **1510**, cutover from the source volume to the destination volume begins by quiescing the source volume. In one embodiment, as part of quiescing the source volume, any outside modification (dirtying) of the container file or any metafiles in the volume directory of the source volume may be precluded. In addition, a CP may be started by the source DEFS (e.g., DEFS **1320a**) hosting the source volume, the container file inode of the source volume may be loaded and further processing may be suspended until the inode is not dirty. Upon resumption, in-core data structures (e.g., a buffer tree) associated with the volume should be invalidated.

[0206] At block **1515**, a message is sent to the destination DEFS with block content of the container file inode of the source volume (the container file inode PVBNs) and reference counts for these PVBNs are updated (e.g., incremented). The message may be sent via a persistent message queue mechanism or via on wire-communication. Additionally, the container file inode of the source volume may be cleared at this point and set to dirty. Another CP may be started at this time for the source DEFS, for example, to clear the disk inode payload in the source inode file.

[0207] At block **1520**, the messages sent by the source node at block **1515** is received at the destination node.

[0208] At block **1525**, the desired volume property metadata associated with the source volume may be copied to the destination. As noted above, the volume properties may be stored external to the container file representing the source volume, for example, in metafiles within the volume directory of the source volume and such copying may involve replication of the metadata or

movement of container file inodes of the metadata files, for example, depending on the size of such metadata files.

[0209] At block **1530**, the PVBNs received in the message from the source node may be moved to the inodes previously created in the destination volume directory. Additionally, the inode count may be updated and the destination volume directory inodes may be marked as non dirtiable.

[0210] At block **1535**, identifiers (IDs) of volume-specific data structures (e.g., the buffer tree ID) may be updated. In one embodiment, the buffer tree ID may be made unique cluster wide by appending the node ID to a monotonically increasing buffer ID portion.

[0211] At block **1540**, the destination volume is mounted and made writable as part of a CP on the destination DEFS. At this stage, the cutover should continue from this point even if there is a reboot.

[0212] At block **1545**, the cluster configuration database is updated to facilitate redirection of traffic to the new destination volume, for example, by the cluster-level volume move job, including switching the volume name and volume ID. In one embodiment, the volume ID of a volume in which a given file is stored may be part of the file handle of the given file. Depending on the particular implementation, the volume ID contained in the file handle, for example, provided with an I/O request by a client may be used by the distributed storage system to direct the I/O request to the appropriate volume, whereas the volume name may be used as a key to identify the appropriate volume in connection with API calls and other workflows. Therefore, by changing the volume name and volume ID of the new destination volume maintained by the distributed storage system to correspond to that of the original source volume transparently causes the client to direct subsequent I/O operations to the new destination volume on the new node. At this point, the cutover from the source volume to the new destination volume is complete. As noted above, since the volume data of the source volume is maintained in place within the storage pool (e.g., storage pod **145**) and need not be copied to move the source volume from the source node to the destination node, the proposed zero-copy volume move may be performed as a constant-time operation, for example, as measured from the beginning of the cutover to the end of the cutover, independent of the size of the volume at issue, thereby reducing downtime and improving the operation of the cluster. In this manner, cutover time is predictable and independent of the amount of data stored on (the size of) the source volume.

[0213] At block **1550**, the source volume is taken offline, the PVBNs within the container file inode of the source volume are removed, and inode block counts are adjusted.

[0214] At block **1555**, the source volume is deleted from the source node by removing it from the source DEFS.

[0215] While in the context of the present example, volume property metadata is described as being copied from the source node to the destination node during cutover, it is to be appreciated all or some portion of the volume property metadata may be performed outside of the cutover, for example, just prior to the cutover.

[0216] While in the context of the flow diagrams of FIGS. **6B** and **9-12** and **14-15** a number of enumerated blocks are included, it is to be understood that examples may include additional blocks before, after, and/or in between the enumerated blocks. Similarly, in some examples, one or more of the enumerated blocks may be omitted and/or performed in a different order.

[0217] Embodiments of the present disclosure include various steps, which have been described above. The steps may be performed by hardware components or may be embodied in machine-executable instructions, which may be used to cause one or more processing resources (e.g., one or more general-purpose or special-purpose processors) programmed with the instructions to perform the steps. Alternatively, depending upon the particular implementation, various steps may be performed by a combination of hardware, software, firmware and/or by human operators.

[0218] Embodiments of the present disclosure may be provided as a computer program product, which may include a non-transitory machine-readable storage medium embodying thereon

instructions, which may be used to program a computer (or other electronic devices) to perform a process. The machine-readable medium may include, but is not limited to, fixed (hard) drives, magnetic tape, floppy diskettes, optical disks, compact disc read-only memories (CD-ROMs), and magneto-optical disks, semiconductor memories, such as ROMs, PROMs, random access memories (RAMs), programmable read-only memories (PROMs), erasable PROMs (EPROMs), electrically erasable PROMs (EEPROMs), flash memory, magnetic or optical cards, or other type of media/machine-readable medium suitable for storing electronic instructions (e.g., computer programming code, such as software or firmware).

[0219] Various methods described herein may be practiced by combining one or more non-transitory machine-readable storage media containing the code according to embodiments of the present disclosure with appropriate special purpose or standard computer hardware to execute the code contained therein. An apparatus for practicing various embodiments of the present disclosure may involve one or more computers (e.g., physical and/or virtual servers) (or one or more processors (e.g., processors **222a-b**) within a single computer) and storage systems containing or having network access to computer program(s) coded in accordance with various methods described herein, and the method steps associated with embodiments of the present disclosure may be accomplished by modules, routines, subroutines, or subparts of a computer program product.

[0220] The term “storage media” as used herein refers to any non-transitory media that store data or instructions that cause a machine to operation in a specific fashion. Such storage media may comprise non-volatile media or volatile media. Non-volatile media includes, for example, optical, magnetic or flash disks, such as storage device (e.g., local storage **230**). Volatile media includes dynamic memory, such as main memory (e.g., memory **224**). Common forms of storage media include, for example, a flexible disk, a hard disk, a solid state drive, a magnetic tape, or any other magnetic data storage medium, a CD-ROM, any other optical data storage medium, any physical medium with patterns of holes, a RAM, a PROM, and EPROM, a FLASH-EPROM, NVRAM, any other memory chip or cartridge.

[0221] Storage media is distinct from but may be used in conjunction with transmission media. Transmission media participates in transferring information between storage media. For example, transmission media includes coaxial cables, copper wire and fiber optics, including the wires that comprise bus (e.g., system bus **223**). Transmission media can also take the form of acoustic or light waves, such as those generated during radio-wave and infra-red data communications.

[0222] Various forms of media may be involved in carrying one or more sequences of one or more instructions to the one or more processors for execution. For example, the instructions may initially be carried on a magnetic disk or solid state drive of a remote computer. The remote computer can load the instructions into its dynamic memory and send the instructions over a telephone line using a modem. A modem local to the computer system can receive the data on the telephone line and use an infra-red transmitter to convert the data to an infra-red signal. An infra-red detector can receive the data carried in the infra-red signal and appropriate circuitry can place the data on bus. Bus carries the data to main memory (e.g., memory **224**), from which the one or more processors retrieve and execute the instructions. The instructions received by main memory may optionally be stored on storage device either before or after execution by the one or more processors.

[0223] All examples and illustrative references are non-limiting and should not be used to limit the applicability of the proposed approach to specific implementations and examples described herein and their equivalents. For simplicity, reference numbers may be repeated between various examples. This repetition is for clarity only and does not dictate a relationship between the respective examples. Finally, in view of this disclosure, particular features described in relation to one aspect or example may be applied to other disclosed aspects or examples of the disclosure, even though not specifically shown in the drawings or described in the text.

[0224] The foregoing outlines features of several examples so that those skilled in the art may better understand the aspects of the present disclosure. Those skilled in the art should appreciate

that they may readily use the present disclosure as a basis for designing or modifying other processes and structures for carrying out the same purposes and/or achieving the same advantages of the examples introduced herein. Those skilled in the art should also realize that such equivalent constructions do not depart from the spirit and scope of the present disclosure, and that they may make various changes, substitutions, and alterations herein without departing from the spirit and scope of the present disclosure.

Claims

1. A method comprising: providing a storage pod having a group of disks containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein an entirety of a global physical volume block number (PVBN) space associated with the storage pod is visible and accessible to all nodes of a plurality of nodes of a cluster representing a distributed storage system; and moving a source volume from a source node of the plurality of nodes of the cluster to a destination node of the plurality of nodes of the cluster without transferring underlying volume data of the source volume by copying file system metadata associated with a first container file representing the source volume to the destination node.
2. The method of claim 1, wherein said moving further comprises creating, by the destination node, a destination volume associated with the destination node.
3. The method of claim 2, wherein the file system metadata comprises content of an index node (inode) of a first container file representing the source volume and wherein said moving comprises copying the content of the inode to a second inode of a second container file representing the destination volume.
4. The method of claim 1, wherein a file system implemented by the source node comprises a write-anywhere file system in which writes are performed to free blocks rather than overwriting existing blocks.
5. The method of claim 1, wherein the file system metadata comprises a PVBN at a top of a node tree representing the source volume and wherein said moving is accomplished by copying the PVBN to the destination node.
6. The method of claim 1, wherein said moving further comprises sending a message identifying a set of one or more PVBNs of a first container file representing the source volume from the source node to the destination node.
7. The method of claim 1, wherein the entirety of the global PVBN space associated with the storage pod is visible and accessible to all nodes of the plurality of nodes of the cluster via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group of disks is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster.
8. The method of claim 7, wherein said moving comprises moving the source volume from a first DEFS of the plurality of DEFSs of the source node of the plurality of nodes of the cluster to a second DEFS of the plurality of DEFSs of the destination node of the plurality of nodes of the cluster by copying an inode of a first container file representing the source volume to a new inode a second container file representing a destination volume within the second DEFS.
9. A distributed storage system comprising: one or more processing resources; and instructions that when executed by the one or more processing resources cause the distributed storage system to: provide a storage pod having a group of disks containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein an entirety of a global physical volume block number (PVBN) space associated with the storage pod is visible and accessible to all nodes of a plurality of nodes of a cluster representing a distributed storage system; and move a source volume from a source node of the plurality of nodes of the cluster to a destination node of the plurality of nodes of the cluster without transferring underlying volume data of the source volume by copying file

system metadata associated with a first container file representing the source volume to the destination node.

10. The distributed storage system of claim 9, wherein moving the source volume includes creating, by the destination node, a destination volume associated with the destination node.

11. The distributed storage system of claim 10, wherein the file system metadata comprises content of an index node (inode) of a first container file representing the source volume and wherein moving the source volume comprises copying the content of the inode to a second inode of a second container file representing the destination volume.

12. The distributed storage system of claim 9, wherein a file system implemented by the source node comprises a write-anywhere file system in which writes are performed to free blocks rather than overwriting existing blocks.

13. The distributed storage system of claim 9, wherein the file system metadata comprises a PVBN at a top of a node tree representing the source volume and wherein moving the source volume is accomplished by copying the PVBN to the destination node.

14. The distributed storage system of claim 9, wherein moving the source volume includes comprises sending a message identifying a set of one or more PVBNs of a first container file representing the source volume from the source node to the destination node.

15. The distributed storage system of claim 9, wherein the entirety of the global PVBN space associated with the storage pod is visible and accessible to all nodes of the plurality of nodes of the cluster via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group of disks is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster.

16. The distributed storage system of claim 15, wherein moving the source volume comprises moving the source volume from a first DEFS of the plurality of DEFSs of the source node of the plurality of nodes of the cluster to a second DEFS of the plurality of DEFSs of the destination node of the plurality of nodes of the cluster by copying an inode of a first container file representing the source volume to a new inode a second container file representing a destination volume within the second DEFS.

17. A non-transitory machine readable medium storing instructions, which when executed by one or more processing resources of a distributed storage system, cause the distributed storage system to: provide a storage pod having a group of disks containing a plurality of Redundant Array of Independent Disks (RAID) groups, wherein an entirety of a global physical volume block number (PVBN) space associated with the storage pod is visible and accessible to all nodes of a plurality of nodes of a cluster representing a distributed storage system; and move a source volume from a source node of the plurality of nodes of the cluster to a destination node of the plurality of nodes of the cluster without transferring underlying volume data of the source volume by copying file system metadata associated with a first container file representing the source volume to the destination node.

18. The non-transitory machine readable medium of claim 17, wherein moving the source volume includes creating, by the destination node, a destination volume associated with the destination node.

19. The non-transitory machine readable medium of claim 18, wherein the file system metadata comprises content of an index node (inode) of a first container file representing the source volume and wherein moving the source volume comprises copying the content of the inode to a second inode of a second container file representing the destination volume.

20. The non-transitory machine readable medium of claim 17, wherein a file system implemented by the source node comprises a write-anywhere file system in which writes are performed to free blocks rather than overwriting existing blocks.

21. The non-transitory machine readable medium of claim 17, wherein the file system metadata

comprises a PVBN at a top of a node tree representing the source volume and wherein moving the source volume is accomplished by copying the PVBN to the destination node.

22. The non-transitory machine readable medium of claim 17, wherein moving the source volume includes comprises sending a message identifying a set of one or more PVBNs of a first container file representing the source volume from the source node to the destination node.

23. The non-transitory machine readable medium of claim 17, wherein the entirety of the global PVBN space associated with the storage pod is visible and accessible to all nodes of the plurality of nodes of the cluster via their respective dynamically extensible file systems (DEFSs) and wherein storage space associated with the group of disks is partitioned into a plurality of allocation areas (AAs), in which a given AA of the plurality of AAs is owned by a given DEFS of a plurality of DEFSs of the cluster.

24. The non-transitory machine readable medium of claim 23, wherein moving the source volume comprises moving the source volume from a first DEFS of the plurality of DEFSs of the source node of the plurality of nodes of the cluster to a second DEFS of the plurality of DEFSs of the destination node of the plurality of nodes of the cluster by copying an inode of a first container file representing the source volume to a new inode a second container file representing a destination volume within the second DEFS.
